Rust And Cracks AI Detection with 3D-Gaussian Splatting

A dissertation submitted in partial fulfilment of

the requirements for the degree of

BACHELOR OF *SCIENCE/ENGINEERING** in Computer Science

in

The Queen's University of Belfast

by

Ze Huei Lim

40443486

26/4/2026

SCHOOL OF ELECTRONICS, ELECTRICAL ENGINEERING and COMPUTER SCIENCE

CSC3002 – COMPUTER SCIENCE PROJECT

Dissertation Cover Sheet

A signed and completed cover sheet must accompany the submission of the Computer Science dissertation submitted for assessment.

Work submitted without a cover sheet will **NOT** be marked.

Student Name: Lim Ze Huei Student Number: 40443486
Project Title: Rust and Crack AI detection with 3D Gaussian Splatting

Supervisor: Dr Mien Van

Declaration of Academic Integrity

Before submitting your dissertation please check that the submission:

1. Has a full bibliography attached laid out according to the guidelines specified in the Student Project Handbook
2. Contains full acknowledgement of all secondary sources used (paper-based and electronic)
3. Contains full acknowledgement of generative AI usage throughout the project
4. Does not exceed the specified page limit
5. Is clearly presented and proof-read
6. Is submitted on, or before, the specified or agreed due date. Late submissions will only be accepted in exceptional circumstances or where a deferment has been granted in advance.

By submitting your dissertation you declare that you have completed the tutorial on referencing and plagiarism at https://www.qub.ac.uk/directorates/sgc/learning/LearningResources/Referencing/ and are aware that it is an academic offence to plagiarise. You declare that the submission is your own original work. No part of it has been submitted for any other assignment and you have acknowledged all written and electronic sources used.
If selected as an exemplar, I agree to allow my dissertation to be used as a sample for future students. (Please delete this if you do not agree.)

Student's signature Lim Ze Huei 26/4/2026

Acknowledgements

Dr Mien Van

Abstract

RotorAI is an AI driven system for automated inspection using video/images analysis and computer vision. It leverages YOLO (You Only Look Once), OpenCV and deep learning frameworks such as TensorFlow and PyTorch to detect and classify surface defects including rust and cracks in real-time. The system processes image or real-time video footage input then performs inference and maps out detected defects which are mapped out onto a 3D Gaussian Splatting model for visualization. The results demonstrate effective and scalable defect detection with improved accuracy and efficiency compared to manual inspection. The project concludes that integrating AI with 3D visualization and cloud deployment significantly enhances industrial maintenance workflows and decision-making processes in an industrial level.

Contents

1.0 Introduction and Problem Area

The reliability and structural integrity of physical assets are critical across a wide range of industrial sectors including manufacturing, energy, transportation and many more. Surface - level defects such as rust and cracks can develop over time leading to reduced performance, structural failure and costly downtime if left undetected. Traditional inspection methods rely heavily on manual visual assessment, which can be time-consuming, subjective and prone to human error, particularly in large-scale or continuous monitoring environments. These challenges have motivated the adoption of automated inspection systems based on computer vision and machine learning techniques.

This project adopts a flexible, vision-based approach to surface defect detection that supports both static image analysis and real-time video processing. Static images allow for detailed inspection of individual surfaces, while continuous video streams enable dynamic monitoring and capture multiple viewpoints of an object over time. This dual-mode capability provides comprehensive surface coverage and makes the systesm applicable across a wider range of inspection scenarios. A deep learning-based object detection model is applied to input data to identify defects such as rust, corrosion, and cracks. In real-time mode, detection is performed on each frame, enabling continuous and scalable inspection in dynamic environments. In static mode, high-resolution images can be analysed for precise and focused defect identification. Image processing techniques are incorporated to enhance visual features and improve detection robustness under varying lighting and surface conditions.

To address the limitations of conventional two-dimensional outputs, the system integrates three-dimensional reconstruction using 3D Gaussian Splatting. Input images whether captured individually or extracted from video streams are utilised not only for detection but also for generating a spatial representation of the inspected object. Detected defects are mapped onto this reconstructed 3D model, providing contextual information about their location and distribution across the surface. This integration enables the transition from isolated detections in individual frames to a unified, spatially aware visualisation, supporting more effective analysis and interpretation.

2.0 System Requirements and Specification

2.1 System Overview

RotorAI is a hybrid inspection system designed to detect surface defects such as rust and cracks using both static images and real time video streams. The system integrates multiple deep learning frameworks to support different modes of the operation: TensorFlow is used for image-based detection, while YOLO implemented PyTorch is used for real-time video analysis.

In addition to 2D defect detection, RotorAI integrates 3D reconstruction using 3D Gaussian Splatting. Detection results obtained from the YOLO pipeline are incorporated into 3D reconstruction process to prove spatial visualisation of detected defects. This enables users to not only identify defects but also understand which position and the distribution of defects across the surface of the object.

The system is implemented as a web-based application, allowing users to upload images or interact with the live camera through the browser interface. RotorAI is intended for both industrial inspection scenarios and general purpose surface analysis, providing accessible and flexible too for defect detection and visualisation to demonstrate complex inspections for non-technical users and expert engineers.

2.2 Assumptions and Constraints

Assumptions:

- Input images are of sufficient quality for defect detection.
- Object being analysed have visible surfaces with minimal occlusion.
- Users have access to a stable internet connection when using the web application.
- System is only used for visual inspection rather than precise physical measurements.
- Pre-computed 3D Gaussian Splatting models are assumed available as sample within the web application for demonstration purposes.

Constraints:

- The system is deployed as a web application and relies on GCP which may introduce limitations in processing speed, availability or request frequency.
- Real-time detection performance is dependent on user's hardware camera and network conditions.
- 3D Gaussian Splatting reconstruction either from videos or images cannot be performed in real time within the web application due to computational complexity, resource limitations and complex boundaries of COLMAPS.

2.3 User Characteristics

The system is designed for a range of users with varying levels of technical expertise:

- **Industrial Inspectors and Engineers:**
  Users with domain knowledge in inspection but limited experience with Artificial Intelligence systems. The interface must therefore be intuitive and require minimal configuration.
- **Researches and Developers:**
  Users with technical backgrounds who may wish to explore the system's capabilities in more detail, including detection outputs and 3D visualisation.
- **General Users:**
  Individuals seeking simple tool for surface detection without prior knowledge of machine learning or computer vision.

Overall the system is designed to be accessible to everyone with a focus on ease of use and minimal learning curve.

2.4 Functional Requirements (FR)

Defect Detection and Analysis

- FR-1: Image upload: The system shall allow users to upload or drag and drop images (JPEG/PNG/JPG) for analysis.
- FR-2: Real-time Camera Feed: The system shall interface with the user's hardware camera to provide a live preview for on-site visual inspection.
- FR-3: Defect Identification: The system will process input frames through Computer Vision Model to identify and highlight surface defects.
- FR-4: Classification: For every detected defect, the system shall provide a classification label.

Visualization

- FR-5: Gaussian Splatting Display: The system shall render a pre-computed 3D Gaussian Splatting model, allowing users to rotate, zoom and pan the model for multi-angle inspection.
- FR-6: Image Display: The system shall provide original image, rust image and cracks image.
- FR-7: Video Display: The system shall provide bounding boxes where rust and cracks are detected.

2.5 Non-Functional Requirements (NFR)

Performance and Scalability

- NFR-1: Latency: The system shall process single image for defect detection in under 3 minutes.
- NFR-2: Rendering Speed: 3D Gaussian Splatting viewer shall maintain a minimum of 30FPS (Frames Per Second) on modern web browsers (Chrome/Safari) with hardware acceleration enabled.
- NFR-3: Page Load: Wen Application shall load and become interactive in under 3 seconds on a standard connection.

Reliability and Security

- **NFR-4: Availability:** The web application hosted Vercel (Frontend) and GCP (Backend) shall maintain uptime of 99.5% during the evaluation period.
- **NFR-5: Data Encryption:** All communication between client and GCP backend shall be encrypted using SECRET KEY AES-256 and TLS 1.2 or higher.

Usability

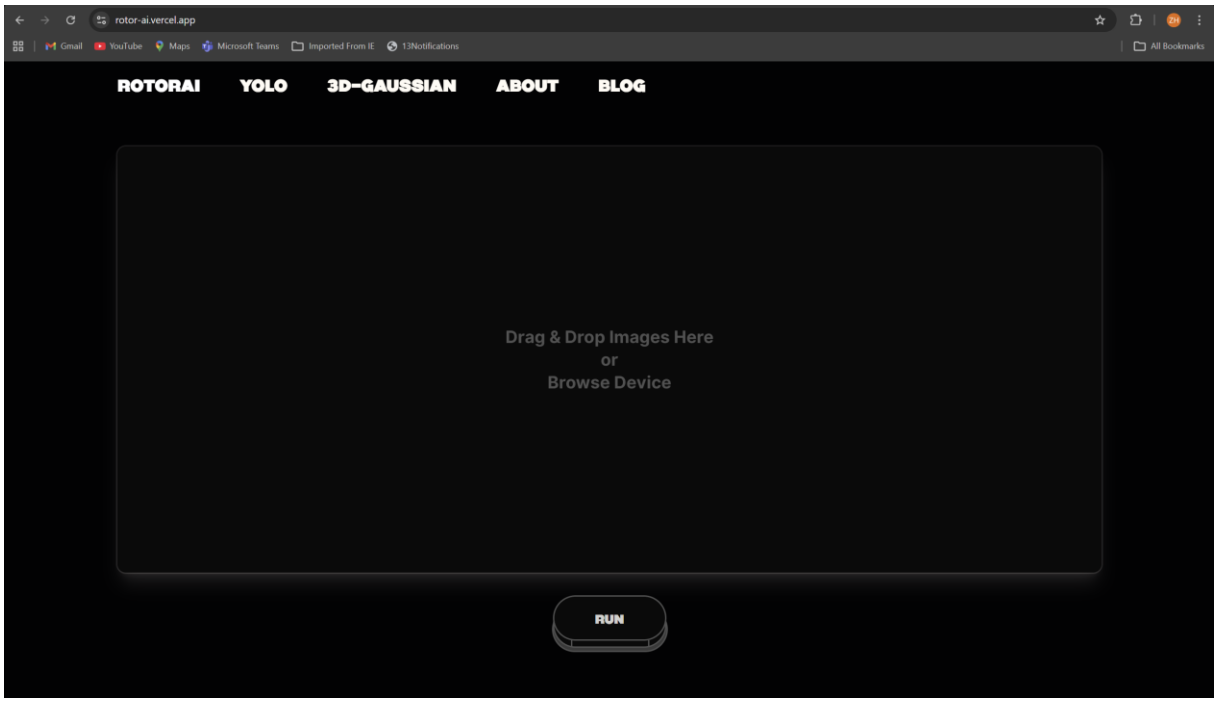
- **NFR-6: Accessibility:** The interface shall achieve a minimum score of 95 and above on the Google Lighthouse Accessibility audit.

3.0 Design

3.1 Architectural Design

RotorAI adopts a modular hybrid architecture designed specifically engineered to support both real-time video processing and static image analysis while enabling scalable deployment and integration with advanced visualisation techniques.

At high level, the system follows a pipeline-based architecture combined with microservice principles where each major functionality is encapsulated as an independent component.



High-Level Architecture Overview

The system consists of the following core components:

1. Inference Plane
    - TensorFlow worker to handle high-resolution defect segmentation
    - A PyTorch-native YOLOv8 implementation optimized for CUDA execution to maintain >30 FPS
2. Spatial Reconstruction Plane
    - Integrate 3D Gaussian Splatting (3DGS)
    - Integrate RaySplat's Intersections for back-projection of 2D bounding boxes into 3D system using camera intrinsics and estimated depth.
3. API Layer
    - Built on Flask/RESTful API using asynchronous worker to bridge TensorFlow and PyTorch runtimes.
    - Encrypted with AES-256 for security.
4. Input Layer
    - Accepts:
        - Static images
        - Real-time video streams (via webcam)
5. Frontend Interface
    - Web-based UI for interaction and visual feedback

Architectural Pattern Justification

This specific pipeline was selected because:

- Supports sequential transformation of data (input -> detection -> refinement -> visualisation)
- Supports Decoupled Multi-Framework Pattern: To bridge TensorFlow and PyTorch as it allows the use of TF's image-processing alongside PyTorch 3DGS and YOLO implementations.
- Prevents UI Freeze during heavy inference spikes.
- Support future scalability and extensibility which also satisfies maintainability.

3.2 Interface Design

UI consists of:

- Upload Interface
    - Enables User to upload static images for analysis.
- Live Detection View
    - Displays real-time video with bounding boxes with highlighted detected defects (Rust and Cracks).
- 3D Visualisation Panel
    - Embedded Gaussian Splat viewer which allows rotation, zoom and spatial inspection.
- Results Dashboard
    - Displays processed outputs

3.3 Software System Design

Input Handler

- Captures Video frames with OpenCV
- Drag and drop images or from folder via API

Detection Modules

Image Detection (TensorFlow)

- Processes single images per request
- Optimised for high-accuracy static analysis

Video Detection (YOLOv8, PyTorch)

- Optimised for speed and real-time inference
- Processes frame sequentially

Post-Processing Module

- Black-Hat Morphological Operations
- Noise filtering
- Region refinement

3D Gaussian Splatting Module

- Integrates detection outputs and original into 3D Space
- Camera poses estimation and Spatial Mapping (RaySplats)

API Layer

- /predict – image inference endpoint (JSON)
- /detect – video streaming detection endpoint (JSON)

Frontend Application

- Built using React
- Integrates Video Rendering and 3D viewer
- Next.js proxy for API endpoints with routing

Data Flow

1. Input
    - Static Image / Frame Capture (OpenCV / NumPy) in base64 and JSON.
2. Analysis
    - Inference TF Image Segmentation / PT YOLO (TensorFlow / PyTorch)
3. Refinement
    - BlackHat Morphological filtering, Confidence Threshold and RaySplats
4. Projection (Only for 3DGS)
    - 3D Coordinate Transformation (Three.js)
5. Output
    - Jsonify JSON output to React Client (GCP / FlaskAPI)

3.4 Key Design and Justification

Hybrid Model Approach (TensorFlow & PyTorch)

Using both TensorFlow (images) and YOLO (video):

- Maximises accuracy (images)
- Maintains real-time performance (video)

Trade-off:

- Deployed with docker as it runs on same but different versions of framework
- Instance based API

Real-Time Processing Constraints

To achieve a good FPS:

- Frame skipping is used
- Frame skipping on motion blur

Concurrency and Event Handling

- **Video pipeline uses frame-by-frame streaming**
- **Non-blocking processing ensures UI responsiveness for Frontend**

Data Persistence

- Stateless API

Future extension

- User Authentication and store history

Error Handling

System handles:

- Model inference failures
- API timeouts

Strategies:

- Fallback responses
- HTTP status codes
-

Security

- API endpoints are structured to call SECRET API KEY before running else would not run
- Deployment with Google Cloud Run with Next.js proxy

Scalability and Deployment

Auto Deployment

- Vercel (Frontend) – push to main triggers automatic CI/CD pipeline, zero downtime deploys.
- Google Cloud Run (Backend) – container-based deployment, automatically builds and deploys new image on push with Cloud Build trigger.

Auto Healing

- Both Vercel and Cloud Run automatically rolls back to last healthy deployment if a deployment fails
- Both platform handle health check without manual intervention

Scalability

- Backend is fully Dockerised making environment reproducible.
- API keys stored as .env variable managed by Secrets Manager (Backend) and .env as sensitive in Vercel (Frontend)
- AI Models (TensorFlow and PyTorch) lives in buckets in Google Cloud Storage allowing model updates without redeployment.
- CORS locked to Vercel domain only restricting unauthorised origins

3.5 Design Evaluation

RotorAI design demonstrates:

- **Strong alignment with system requirements**
- **Effective balance between accuracy and performance**
- **Compatibility with future extensions such as additional defect types or analytics**

4.0 Implementation

4.1 Choice of Implementation and Development Environment

RotorAI was implemented using a combination of Python and JavaScript for machine learning performance and frontend interactivity.

Programming Languages

- Python
    - Used for backend development, machine learning and image/video processing
- JavaScript (React)
    - Used for frontend development
    - Enables interactive, real-time user interfaces with Tailwind CSS and Framer Motion

IDE and VCS

- VS Code and PyCharm were used IDE and GitHub for code management
- Docker – containerising for Cloud Run

Multi Environment Setup

RotorAI was developed using 2 separate Python environments to manage dependencies effectively:

- A PyTorch-based environment with Anaconda/miniconda3 for real-time detection, 3D reconstruction and Gaussian Splatting.
- A TensorFlow-based environment for image classification and segmentation tasks

The separation was necessary due to:

- Conflicting dependencies between TensorFlow and PyTorch
- Different CUDA version requirements

This design improved:

- Stability of the system
- Reproducibility of experiments
- Ease of debugging

4.2 Software Libraries and Tools

RotorAI was implemented using a wide range of libraries to support machine learning, computer vision, 3D recontruction and system deployment. Rather than relying on a single framework, the system integrates multiple specialised libraries to optimise performance across different tasks.

Deep Learning Frameworks

- **TensorFlow (with Keras)**
  - Used for static image-based defect detection
  - Provides stable deployment for trained models
- **PyTorch**
  - Used for real-time object detection with YOLOv8
  - Selected for its flexibility and strong GPU support when training models

Computer Vision and Image Processing

- **OpenCV**
  - Handles video capture, frames extraction and preprocessing
  - Essential for real-time implementation
- **Scikit-image**
  - Used for image filtering and enhancement during post-processing
- **NumPy**
  - Core numerical library for array manipulation and data handling
- **Pillow (PIL)**
  - Image conversion

Object Detection and AI Models

- **Segmentation-models / EfficientNet / Unet / ResNet50**
  - Used in TensorFlow pipeline for image-based defect detection
- **Ultralytic YOLOv8 (medium model)**
  - Primary model for real-time rust detection
  - Provides bounding box prediction and confidence scoring

3D Reconstruction and Visualisation

- **NerfStudio**
    - Used for neural radiance field (NeRF) processing and scene reconstruction
- **Gsplats (Gaussian Splatting)**
    - Enabales efficient real-time 3D rendering of reconstructed scenes
- **Open3D**
    - Supports 3D data processing and point cloud manipulation
- **PyCOLMAP / HLOC**
    - Used for camera pose estimation and feature matching

GPU Acceleration and Performance

- **Ubuntu (Linux)**
    - Environment setter for libraires and AI trainings
- **CUDA Toolkit (cu12 / cu11)**
    - Enables GPU-accelerated computation (RTX 5070)
- **cuDNN, cuBLAS, TensorRT**
    - Optimise deep learning inference performance

Data Handling and Anlaysis

- **Pandas / Polars**
    - Used for structured data processing and logging
- **Matplotlib / Plotly**
    - Used for visualisation and debugging

Backend and API Development

- **Flask / Next.js**
    - Used to implement RESTful API endpoints
- **HTTP Libraries**
    - Enable communication between frontend and backend
- **Base64 / BytesIO**

- Used for image encoding in API responses to decrease the size of POST and GET responses.

Development and Experimentation Tools

- **Jupyter Notebook / IPython**
  - Used for prototyping and experimentation
- **Weights & Biases / Coment ML**
  - Used for experiment tracking and model evaluation

4.3 Key Implementation Decisions

Separation of Image and Video Pipelines

Instead of using a single model for all inputs, two specialised pipelines were implemented:

- TensorFlow – static images (higher accuracy)
- YOLOv8 – real time video (higher speed)

This improve the overall system performance and avoids compromising between speed and accuracy as YOLOv8 model is much suitable for real time whereas TensorFlow model provides heat maps that fill the area rather than bounding boxes.

Frame-based Video Processing

Video streams are process frame-by-frame using OpenCV.

Optimisations include:

- Frame skipping every 5 Frames to maintain FPS
- Resizing frames before inference as resolutions like 4k may take long time to process
- Efficient memory reuse

GPU Acceleration

CUDA-enabled GPU used for PyTorch inference which significantly improves real-time detection speed.

Stateless API Design

Backend API does not have store session data nor a database where each request is processed independently.

Benefits:

- Simplifies deployment
- Enables horizontal scaling

4.4 Implementation of Key Functions and Algorithms

Real-Time Detection Pipeline

1. Capture frame using OpenCV
2. Preprocess frame (resize, normalize)
3. Pass frame to YOLO model
4. Extract bounding boxes and confidence scores
5. Apply post-processing filters
6. Render detections on frame

This pipeline ensures low latency and continuous processing avoiding UI freezes or crashes.

Image-Based Detection

For Static Images:

- Input image is passed through TensorFlow model
- Output is processed similarly to video detections
- Higher Resolution processing is used for improved accuracy

Pre-processing Algorithm

The custom function apply_clahe enhances raw images before model outputs.

Algorithm:

- Adding contrast enhancement of 1.5 before inference

Purpose:

- Improving defect detection for low-light/poor quality footage

Post-Processing Algorithm

The custom function detect_rust_and_cracks enhances raw model outputs.

Algorithm:

- BlackHat Morphological operations
- Noise Reduction using filtering
- Region-based Refinement

Purpose:

- Reduce False Positives
- Improve defect boundary accuracy

3D Gaussian Splatting

- Detection coordinates are mapped into 3D space
- Combined with camera pose data
- Rendered in browser using WebGL ((@mkkellogg/gaussian-splats-3d) and Three.js

Ray-Splat Intersection:

- Used RaySplat Sphere Algorithm to map out defect detections on images.
- Images is then compared to NeRF .ply and creates a JSON file of where is the defects are and maps using Three.js and WebGL 3DGS.

4.5 Scalability and Deployment

Containerisation

This was necessary due to the complex dependency requirement of running both TensorFlow and PyTorch in the same environment.

The Docker image is configured to:

- Install all Python dependencies from requirements.txt
- Expose port 8080 for Cloud Run compatibility
- Set environment variables for framework configuration (e.g. SM_FRAMEWORK=tf.keras)

Model Storage

ML models are stored in Google Cloud Storage rather than inside the container to keep the image lightweight. On startup, download_model() checks if models already exist in /tmp before downloading, avoiding redundant downloads on warm instances.

5.0 Testing

5.1 Test Approach

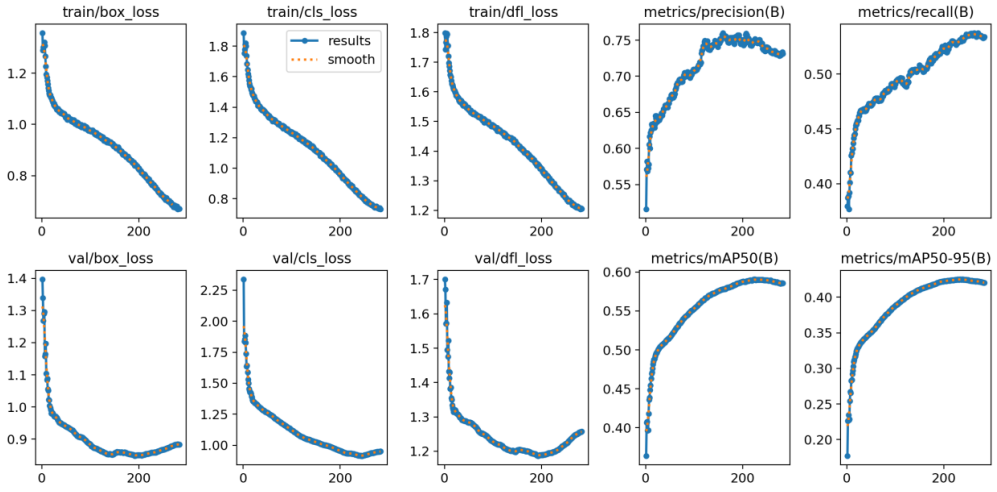
Three levels of testing were conducted:

- Model Evaluation Testing – assessing ML model performance using standard metrics
- API Testing – validating endpoint behaviour using Postman and GCP
- System Testing – end-to-end pipeline validation

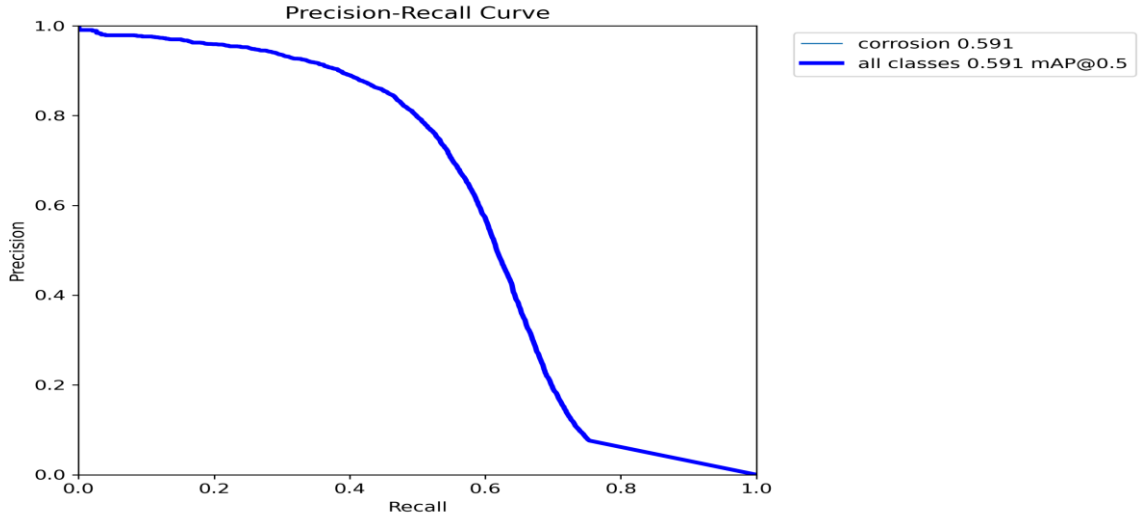
5.2 Model Evaluation Testing

AI Models were evaluated using

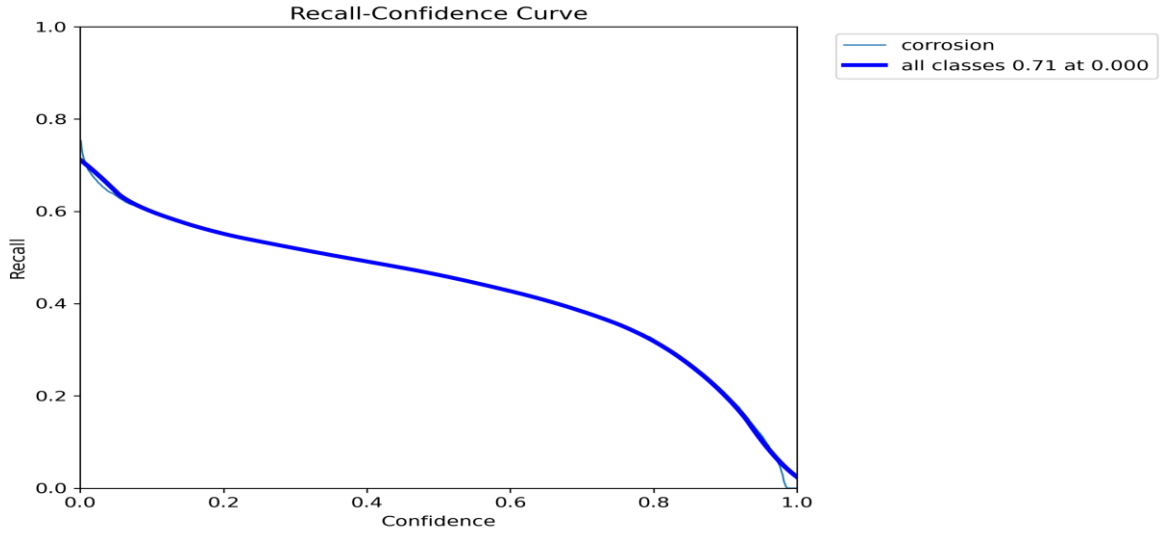
- Precision / Recall / F1– to measure defect classification accuracy
- Loss Curves – to validate training convergence and detect overfitting



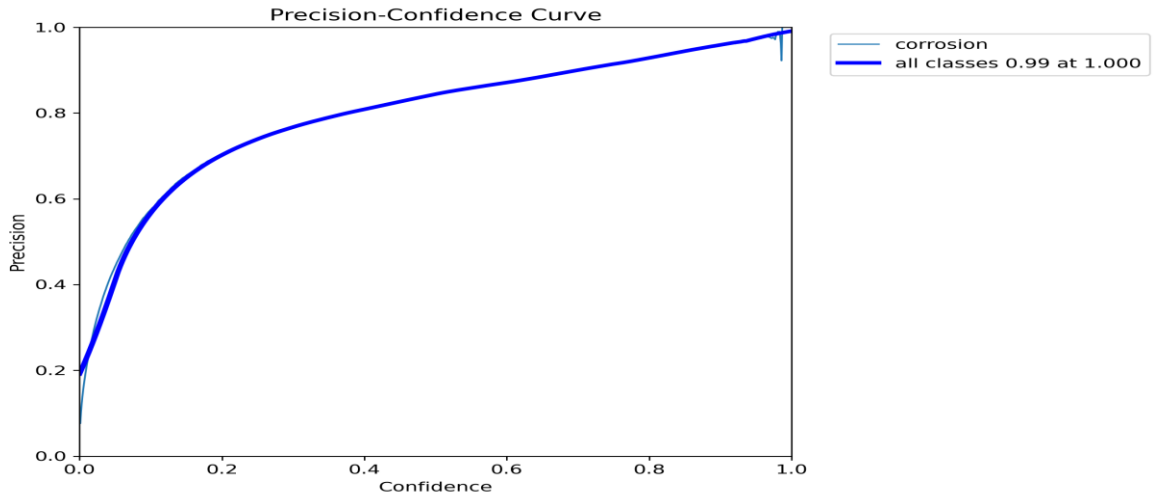
Precision-Recall Curve:



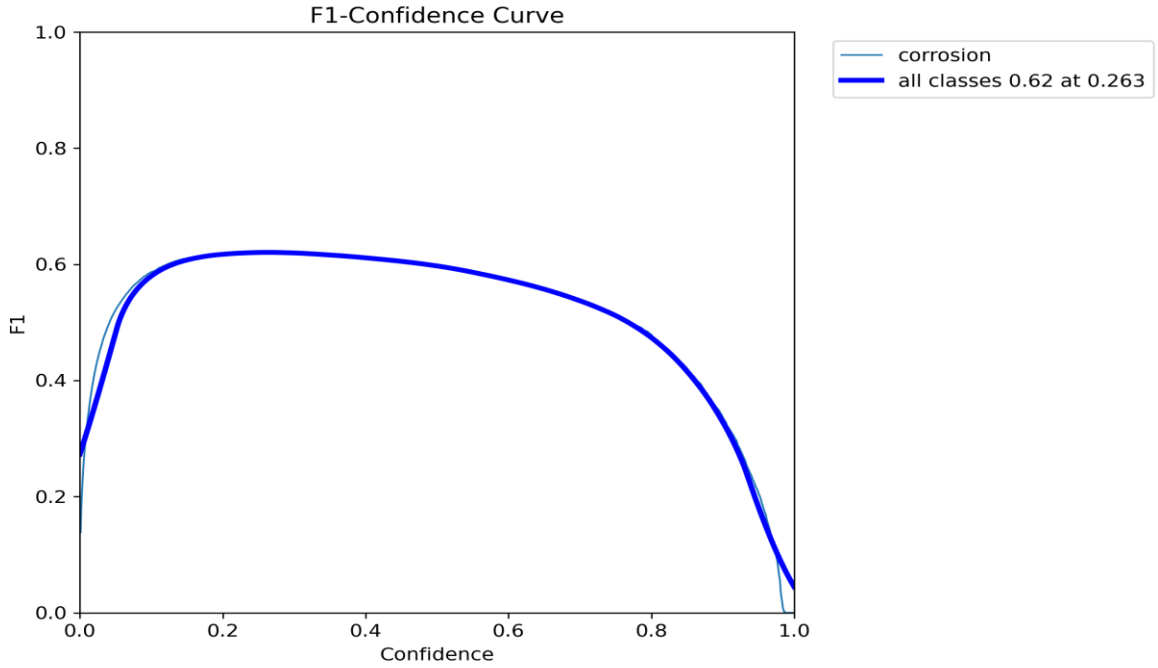
Recall-Confidence Curve:



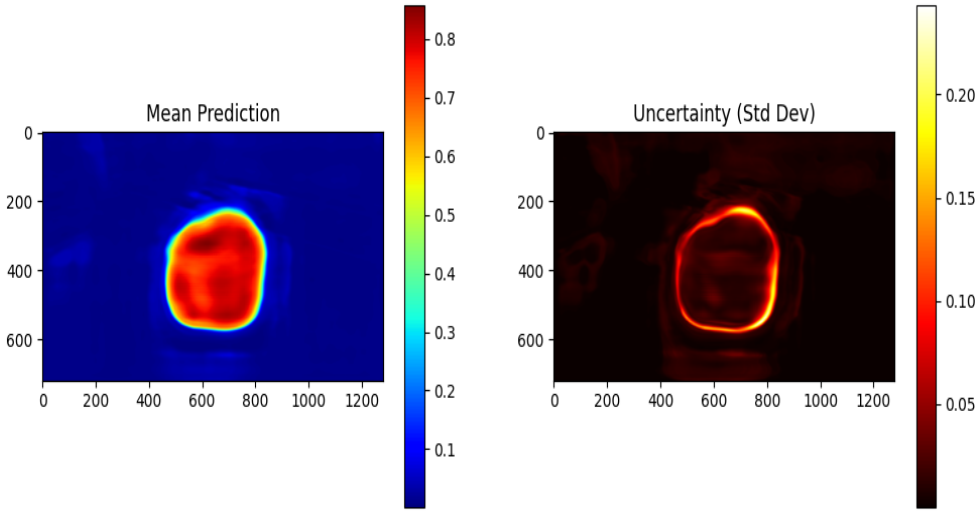
Precision-Confidence Curve:



F1-Confidence Curve:

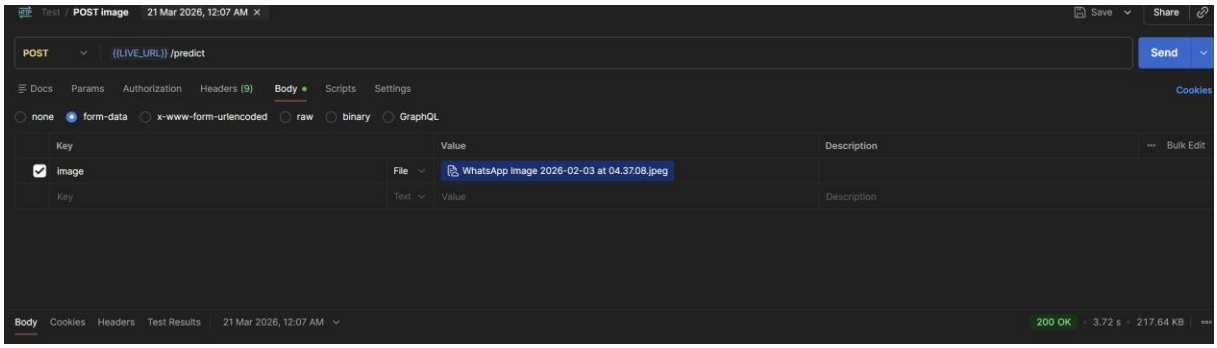


Monte-Carlo Testing:



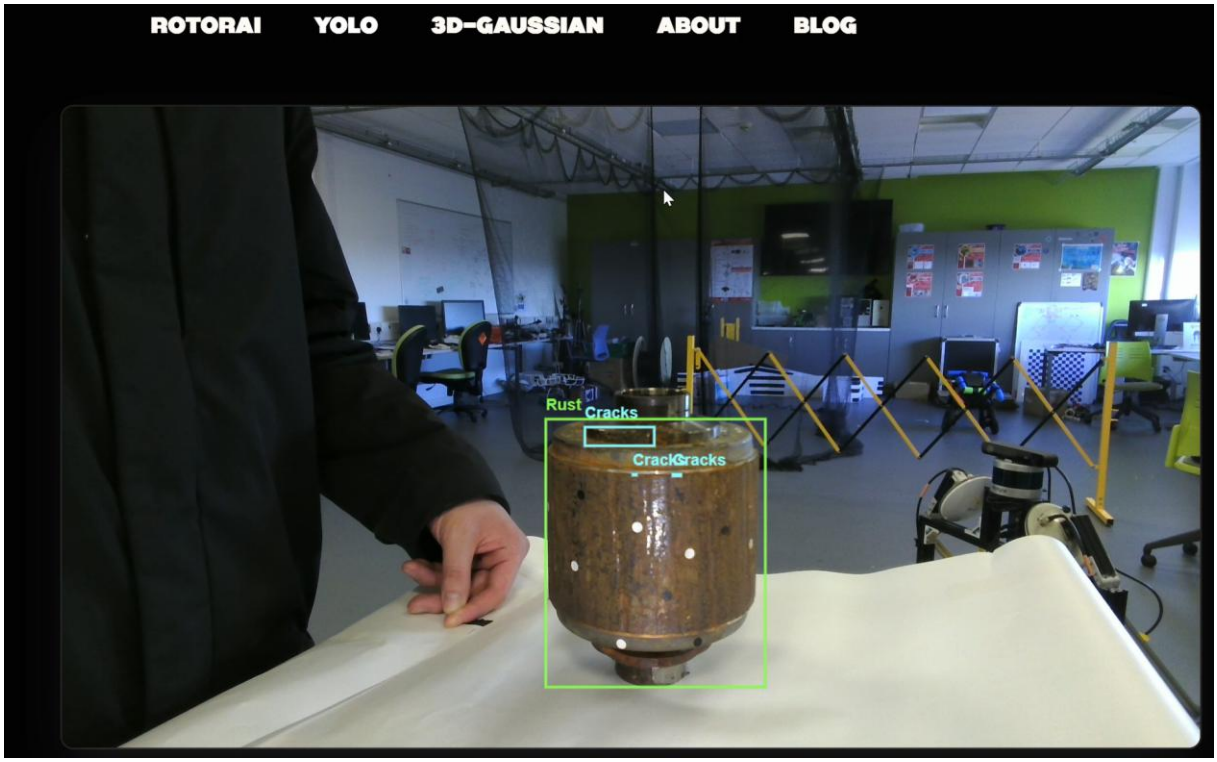
5.3 API Testing (POSTMAN & GCP)

Endpoints tested: /predict



5.4 System Testing

End-to-end testing pipeline: /detect



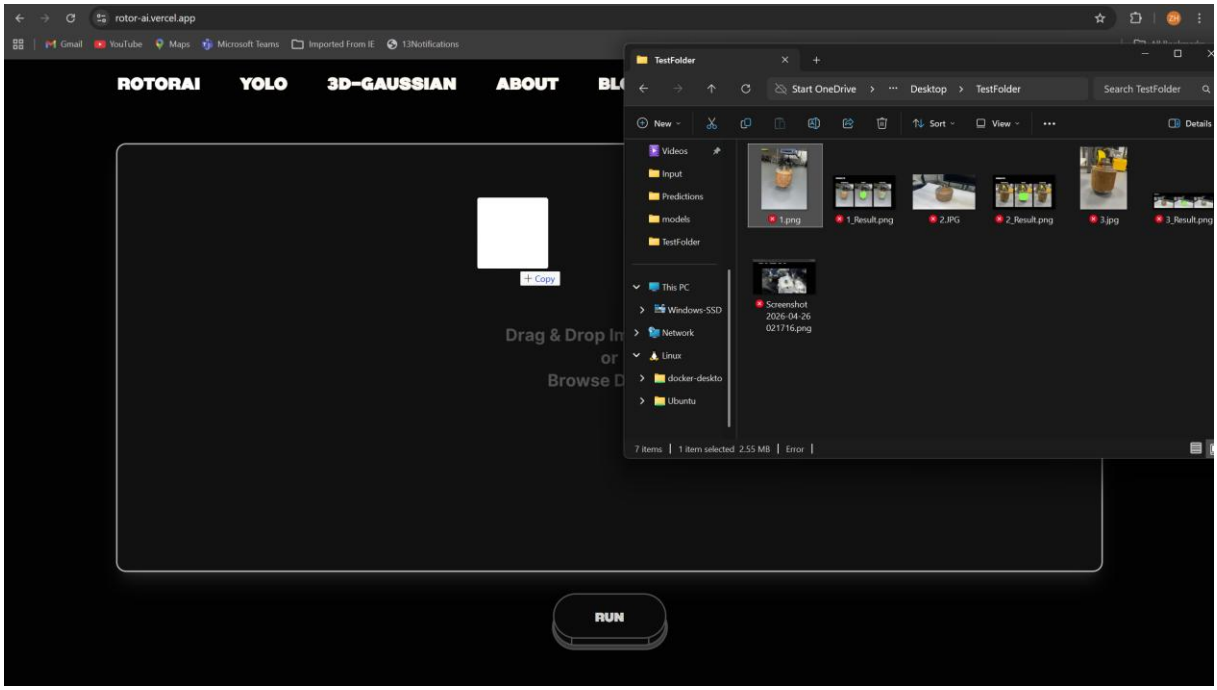
Note: Detection depends fully on camera quality as lower quality results in lesser focused defect detections due to high confidence threshold at 0.65.

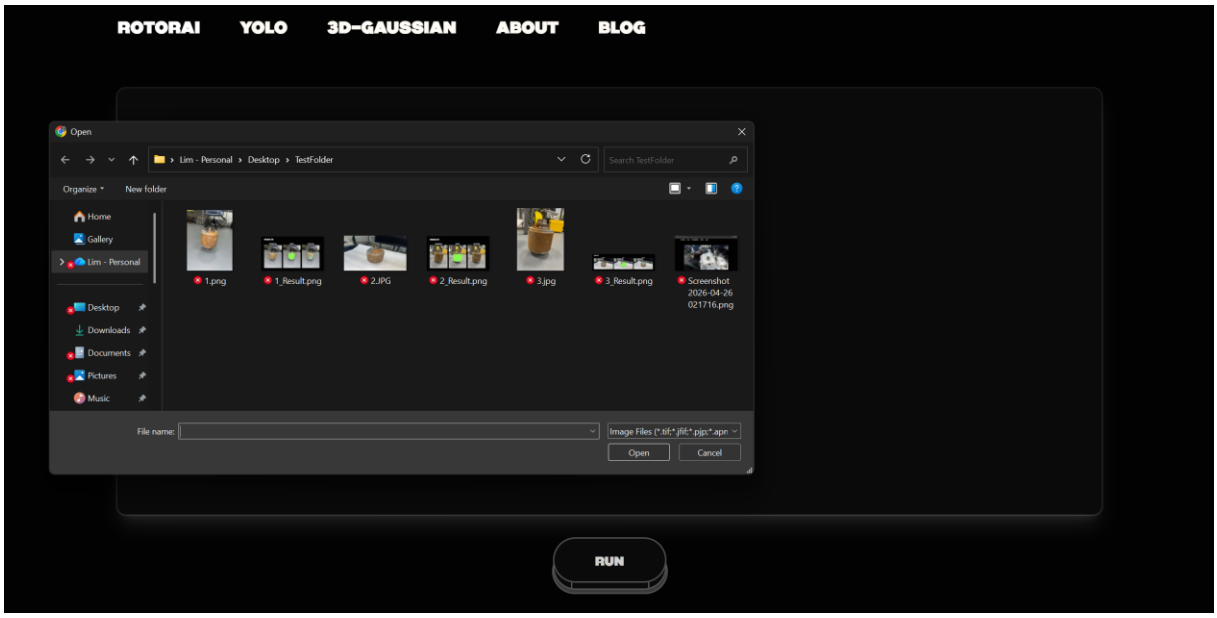
6.0 System Evaluation and Experimental Results

6.1 Evaluation Against Requirements

Functional Requirement 1: Image upload:

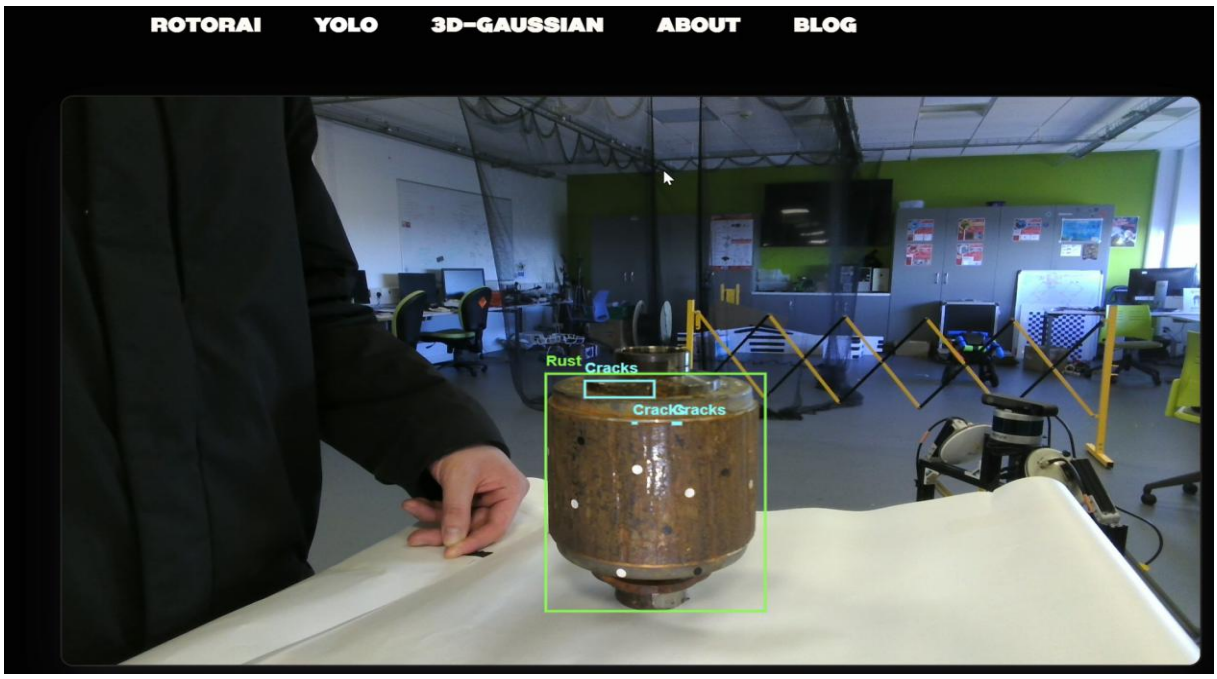
Image upload from devices or drag and drop works perfectly in the website.





Functional Requirement 2: Real-time Camera Feed:

Camera feed is able to send frames and maintain in 60 FPS and above



Functional Requirement 3: Defect Identification:

The feature works as all rust output images will be Green masks or boxes and cracks output image will be in Cyan masks or boxes.

Functional Requirement 4: Classification:

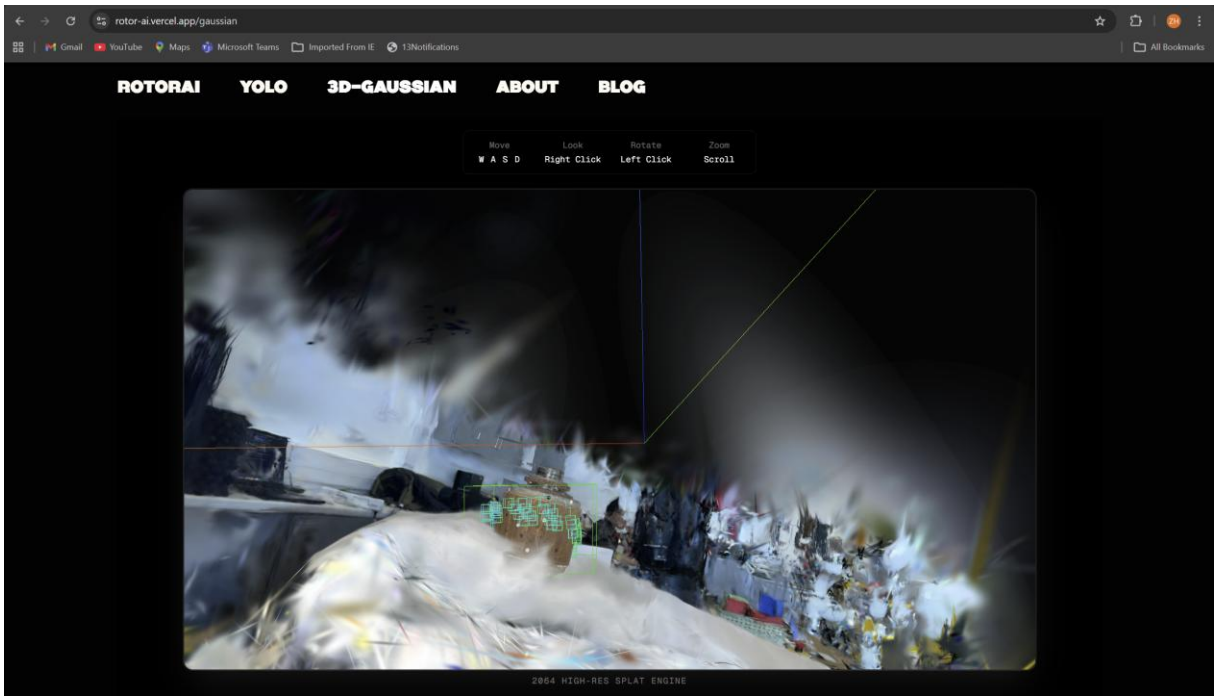
The system successfully classifies detected defects into two distinct categories:

- **Rust — highlighted with a green overlay on static images and green bounding boxes in real-time video**
- **Cracks — highlighted with a cyan overlay on static images and cyan bounding boxes in real-time video**

Both the image pipeline and video pipeline detect and classify rust and cracks independently. In the video pipeline, each detection is accompanied by a confidence score displayed alongside the bounding box, allowing users to assess the reliability of each classification. Both defect types are returned as separate outputs in the API response, enabling clear distinction between rust and crack detections in the results dashboard.

Functional Requirement 5: 3D Gaussian Splatting Display:

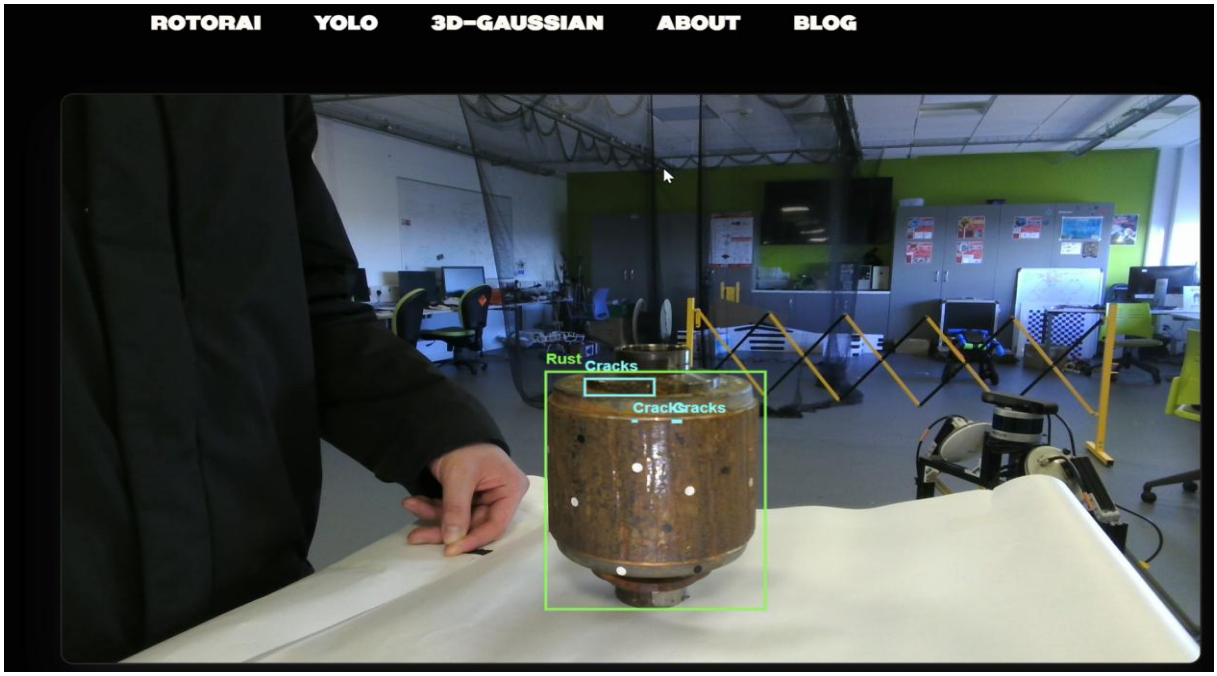
3D Gaussian Splatting Display is able to render in 2064px (high quality) with interactive interface for users to move around.



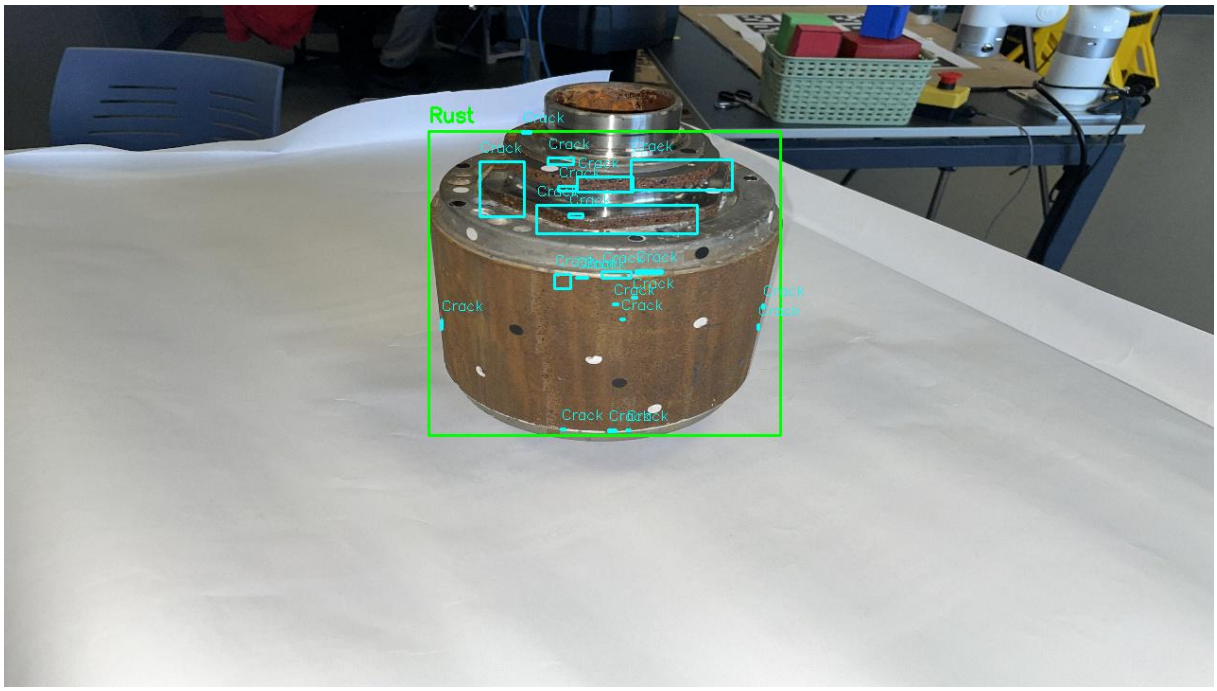
Functional Requirement 6: Image Display:

Functional Requirement 7: Video Display:

60FPS – Internal Web Camera



120FPS – External Web Camera



Non- Functional Requirement 1: Latency:

Image processing was completed within the 3 minute threshold under normal conditions. However cold start latency on Cloud Run occasionally extended initial response times beyond this threshold.

Non- Functional Requirement 2: Rendering Speed:

The 3DGS viewer is being to run at a high quality on modern browsers.

Non- Functional Requirement 3: Page Load:

Page Load Google Lighthouse recorded a performance score of 100, confirming the web application loads and becomes interactive well within the 3 second threshold.

Non- Functional Requirement 4: Availability:

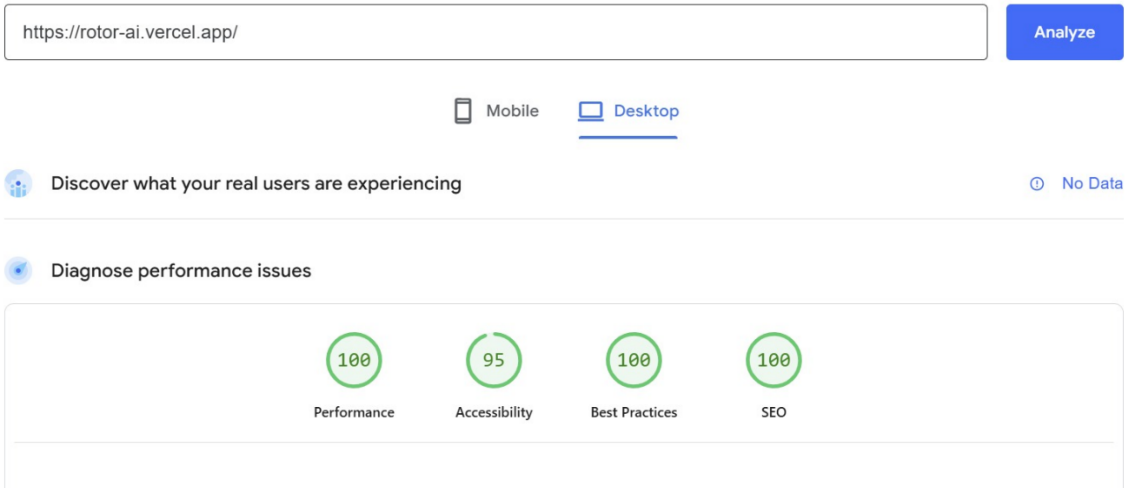
Both Vercel and Google Cloud Run maintained uptime throughout the evaluation period with no recorded outages.

Non- Functional Requirement 5: Data Encryption:

All client backend is encrypted with SECRET MANAGER with AES-256 encryption and is authenticated before executing. Frontend is first communicated with Next.js proxy before sending request to Backend.

Non- Functional Requirement 6: Accessibility:

Google Lighthouse recorded an accessibility score of 95, meeting the minimum threshold of 95 defined in the requirements.

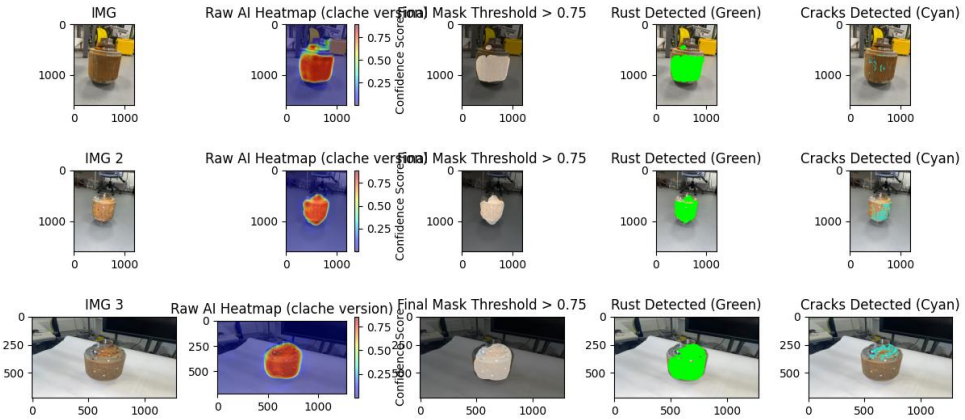


6.2 Experimental Results

Both Model uses the same datasets for training with about 40,000 images.

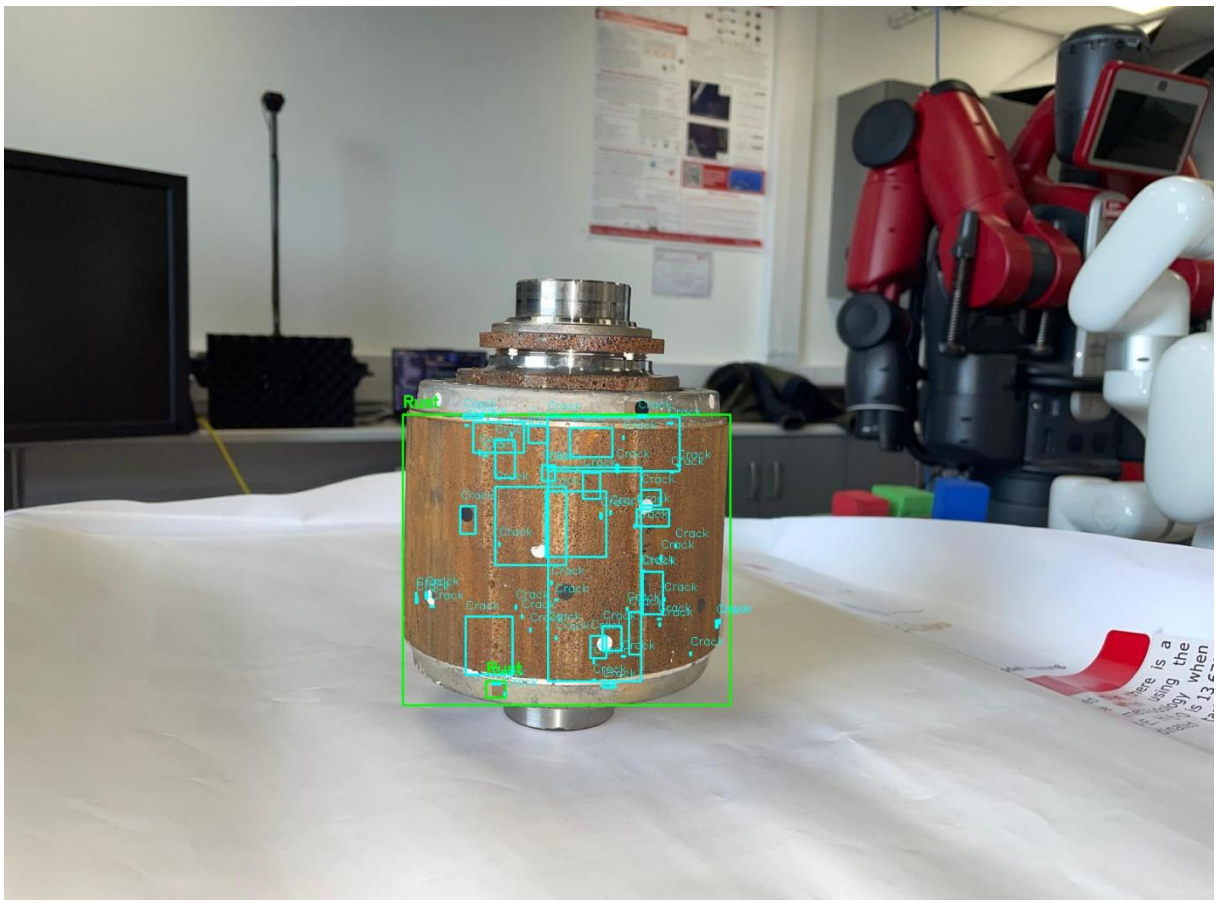
AI Model Performance

Final TensorFlow Results:

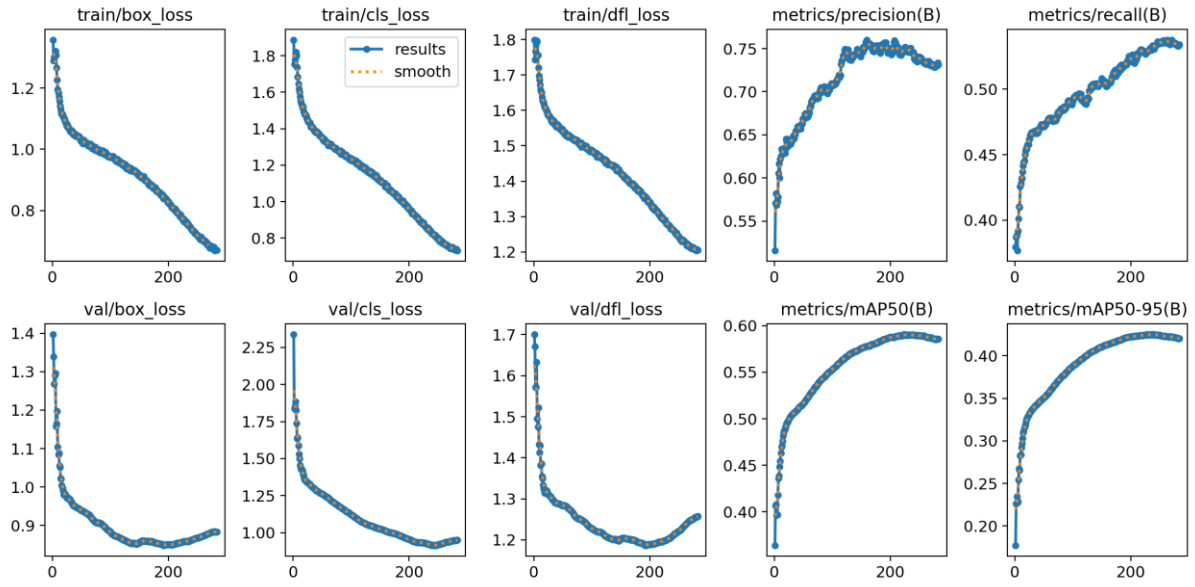


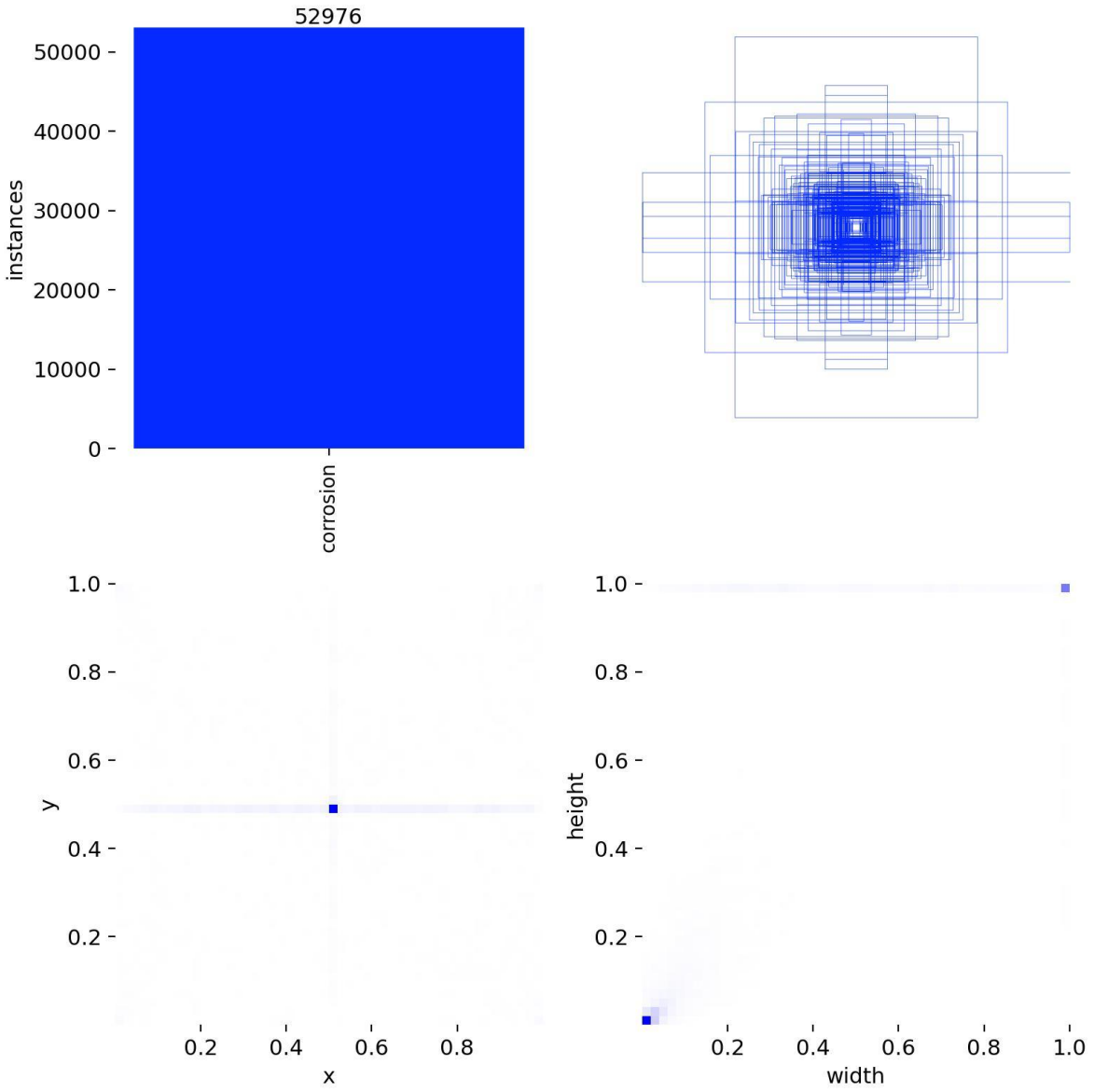
Final PyTorch Results:

https://github.com/ZEHUEI/RotorAI-zip - Full Video Here



AI Training Data:



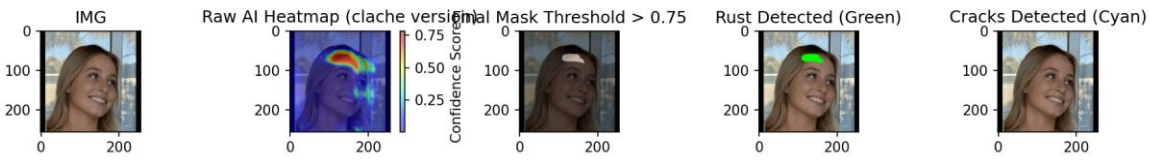


An improvement for this is to add data with rust at the corners so AI model wont be at the middle of (0.5, 0.5) in x against y graph.

Failed examples and justification changes:

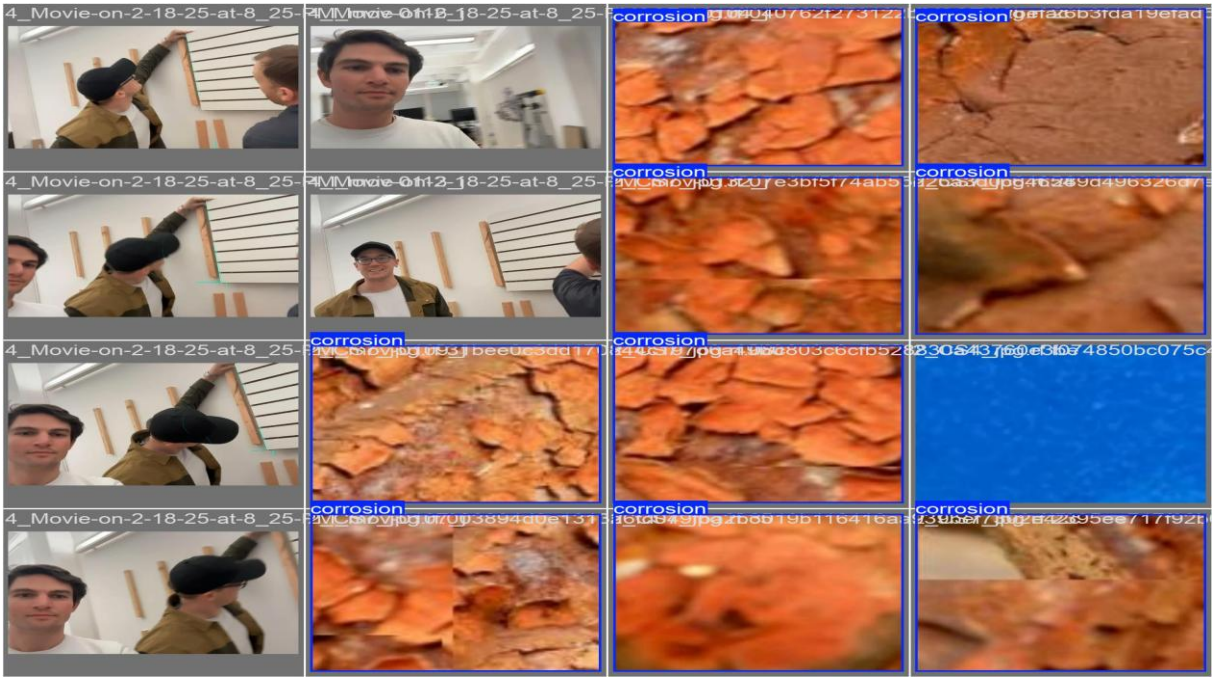
Models are scanning dirt patches and human faces / hair (brunette color) as rust and cracks.

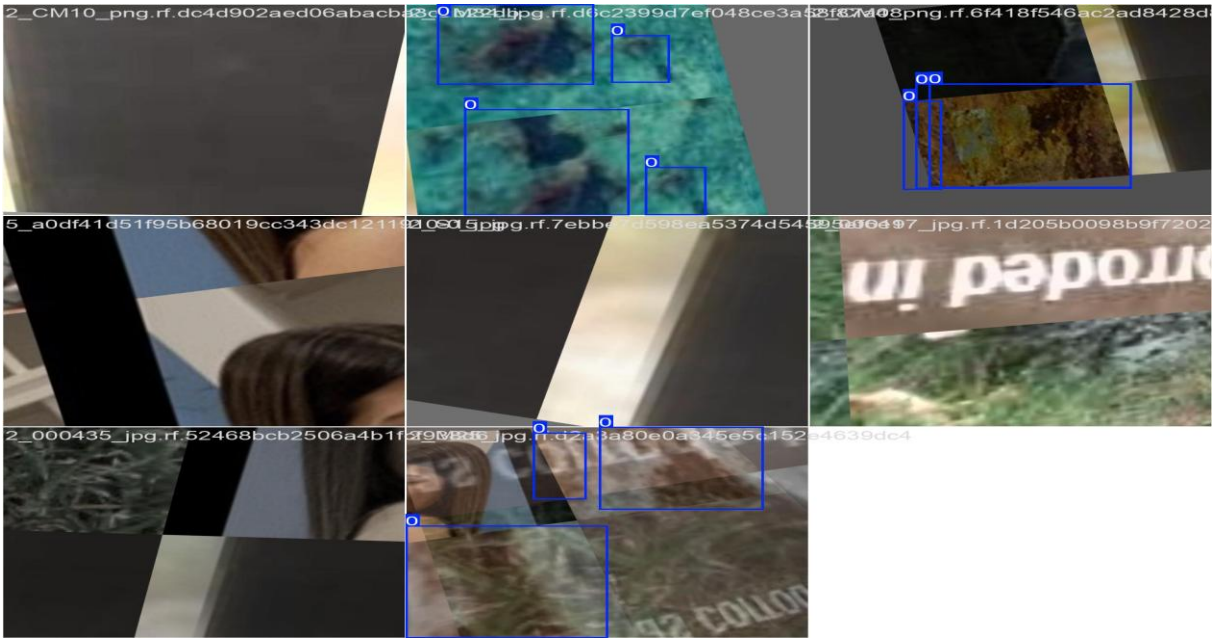
Old Model:



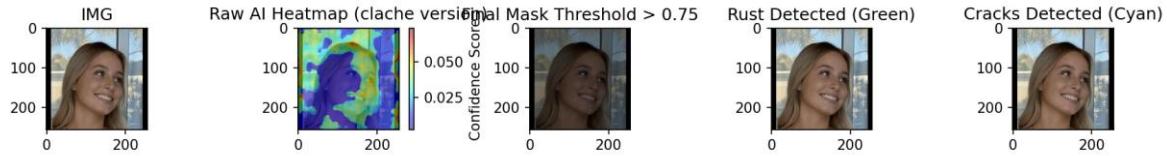
Justification:

Extra data of human and background are added as false negatives where AI models retrain and are able to classify them properly.





Final Model:



3D Gaussian Splatting Results:

How 3DGS Works

3D Gaussian Splatting begins with sparse point cloud generated from multi view images using structure from motion (COLMAP). Each point is then represented as an ellipsoidal primitive with properties including position, colour, opacity and covariance. These Gaussians are optimised through differentiable rasterization to best reconstruct the appearance of the scene from any viewpoint, enabling real-time novel view synthesis without baking into a fixed representation.

3D Gaussian Splatting builds upon the foundation established by Neural Radiance Field (NeRF) research which pioneered the use of neural networks for photorealistic 3D scene reconstruction from multi-view images or videos. Conceptually, 3D Gaussian Splatting can be understood as stitching hundreds of overlapping images captured from different viewpoints.

Reconstruction Quality

The pre-computed 3D Gaussian Splatting model accurately represents surface geometry and appearance allowing users to inspect defect distribution across the structure from multiple angles. The viewer supports rotation, zoom and pan.

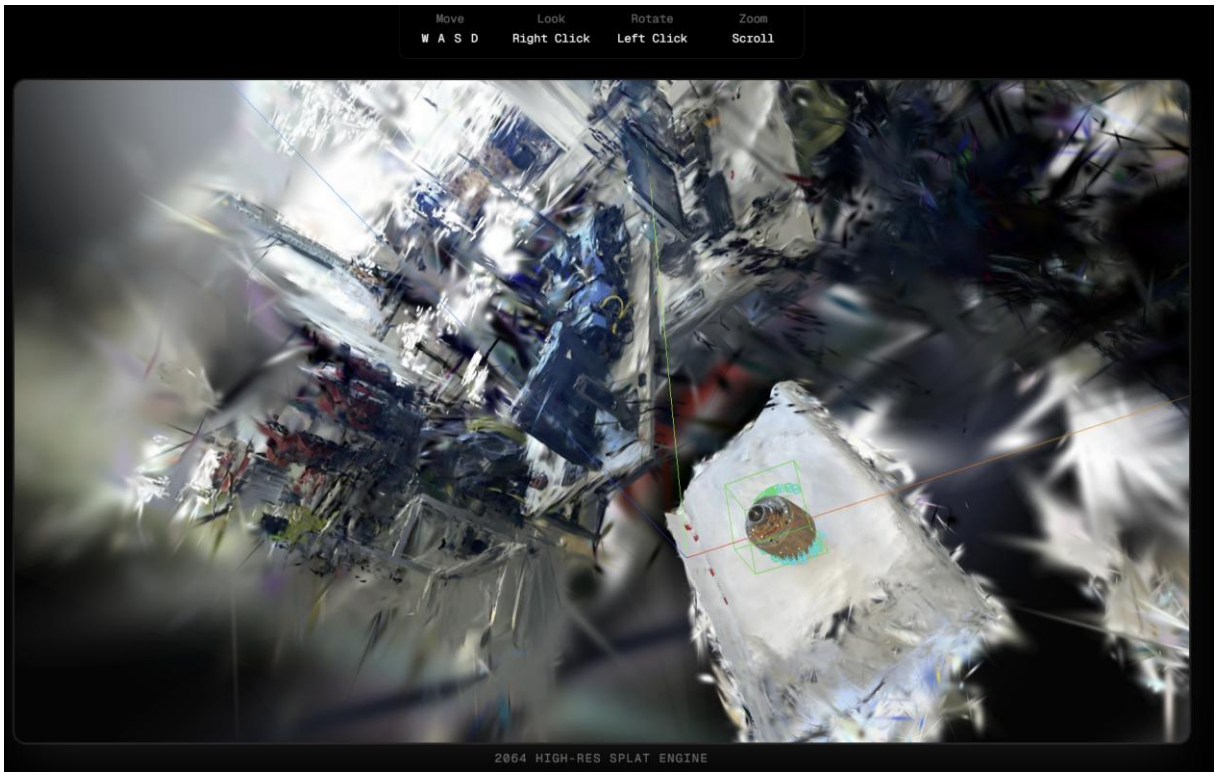
Rendering Performance

The 3D Gaussian Splatting achieved a rendering speed of 2064 FPS high resolution in the web browser using WebGL. This demonstrates that browser-based 3D rendering of Gsplats scene is achievable without dedicated GPU hardware on client side.

Limitations:

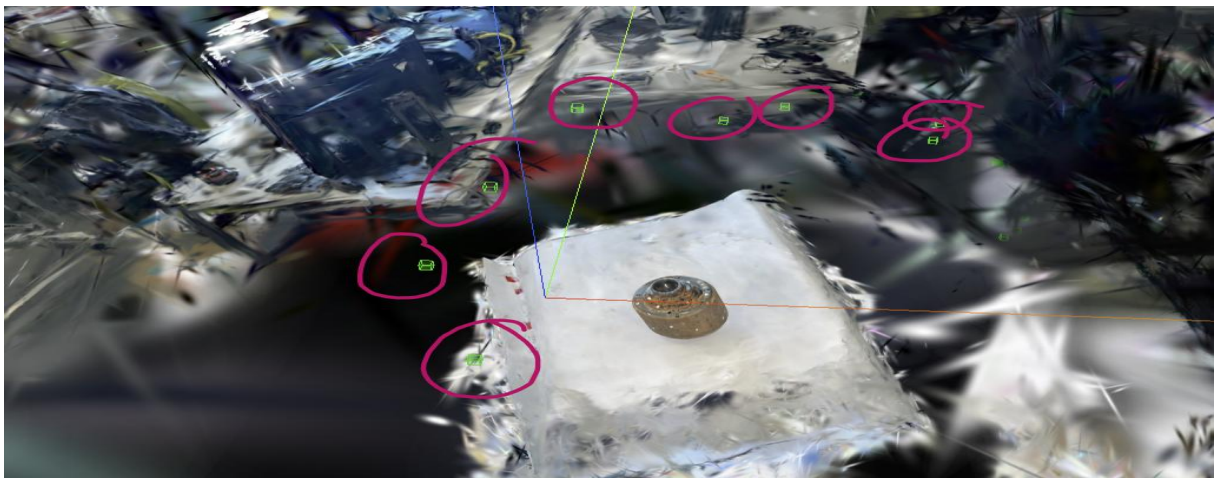
- **Reconstruction quality depends heavily on number and quality of input images / video**
- **Thin surfaces or reflective surfaces reconstruct poorly**
- **Cannot run reconstruction in real-time, only pre-computed models are supported as it will take long hours.**

3DGS Results (Ashby Building LVL10):

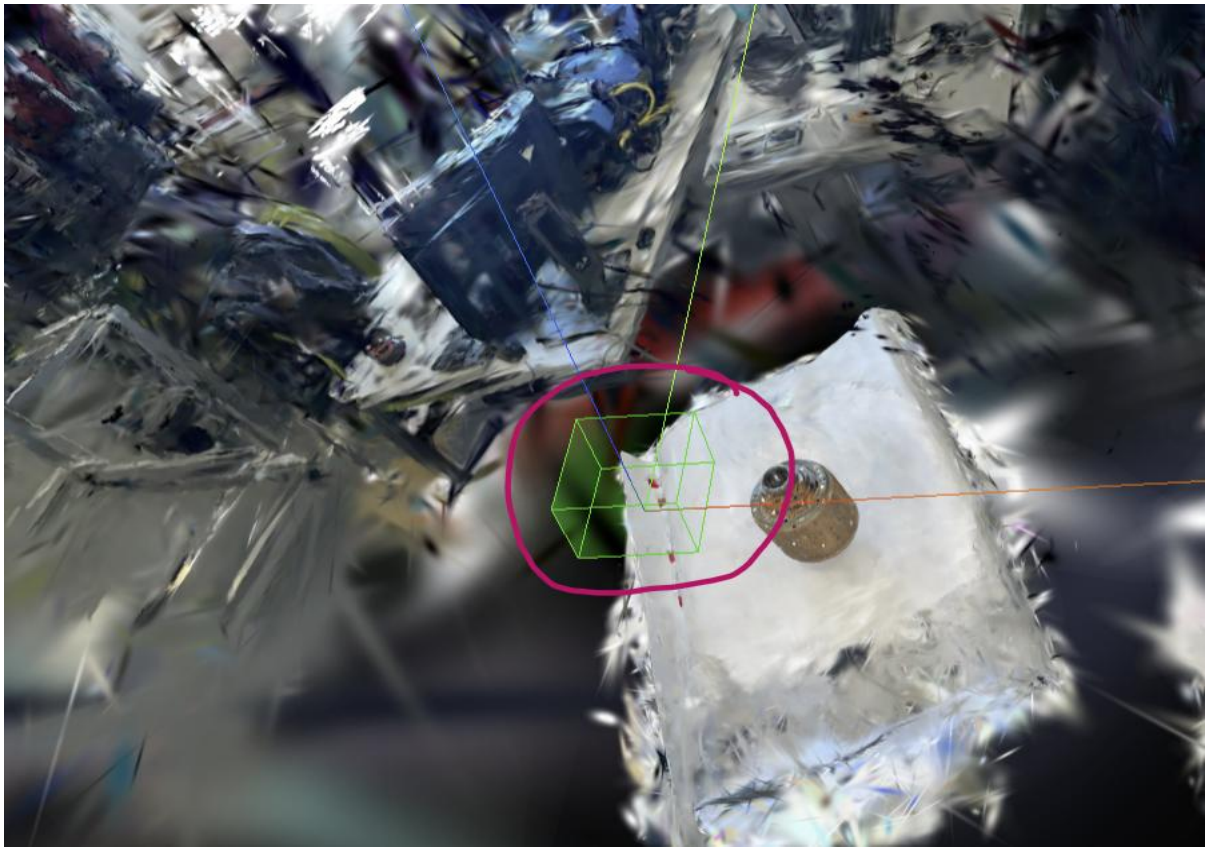


RaySplat Intersection Results:

Standard ray-splat intersection maps 2D detections into 3D space by casting rays through flat Gaussian planes. However this approach produced inaccurate results for curved and irregular surfaces such as rotor components, where flat plane assumptions caused misalignment with the actual surface geometry. To address this, a ray-sphere intersection approach was adopted, which models the projection surface as a sphere rather than a flat plane, better approximating the curved geometry of inspected components.

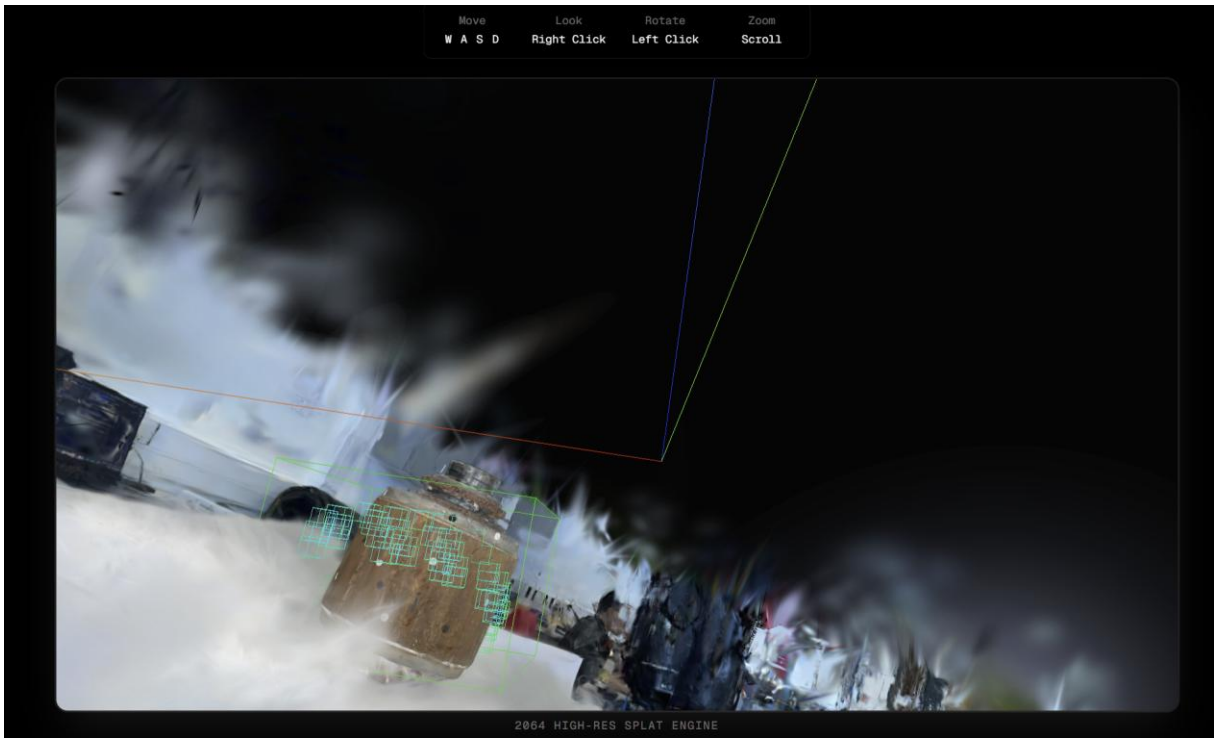


As shown in the figure above, the reason why a ray-sphere method was used instead of rectangle or a cylinder was because the images was taken 270 degrees and the sphere results returns the best. This figure projects 2D bounding boxes from each image viewpoint into 3D space, resulting in multiple small individual boxes distributed across the scene from their respective camera positions.



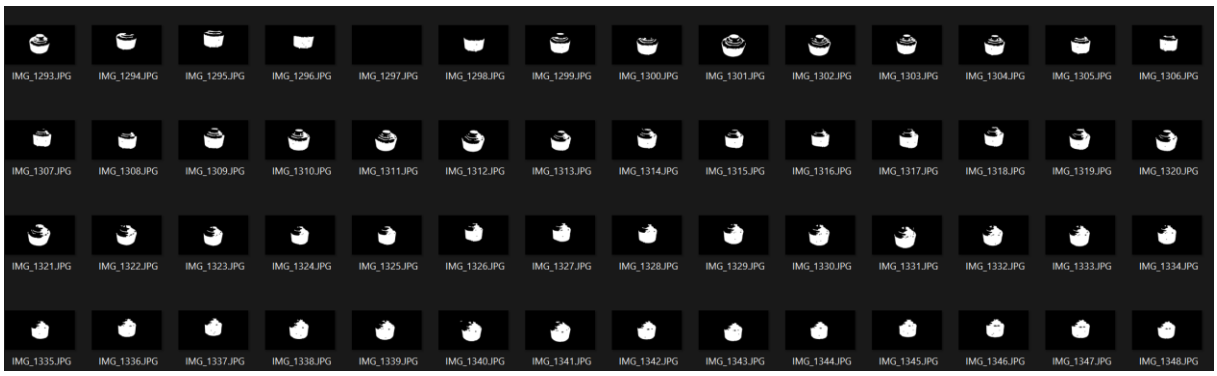
These individual viewpoint boxes are then merged into a single unified bounding volume as shown in the figure below, representing the overall rust regions as one coherent area from each camera angle. This merging step reduces visual clutter and provides clearer spatial understanding of where rust is on. The box is also at position x = 0, y =0 and z =0.

Finally, for crack detection the bounding boxes remain individual and are not merged as cracks are thin and distributed across distinct locations where merging them will results in worse accuracy as shown in the figure below.



Justification

The proposed method at the start of the project was using masks as overlay in 3D Gaussian Splatting however this method would not work as it will hurt the outcome of 3D Gaussian Splatting. This method is done by getting the AI model to return 0 and 1 masks then putting it into transform.json and reconstruct 3D Gaussian Splatting.



However, this method was abandoned as injecting masked images into reconstruction will significantly degrade the final quality of 3DGS scene. Since 3DGS relies on photometric consistency across all input, introducing binary masks disrupted this causing reconstruction to produce poor visual quality and inaccurate geometry.

As a result, ray-spehere intersection approach is adopted instead keeping 3DGS reconstruction separated from AI detection pipeline.

6.3 Societal and Commercial Impact

Industrial Inspection

Traditional structural inspection such as wind turbines, buildings and many more requires trained personnel to physically access dangerous or hard-to-reach surfaces. RotorAI demonstrates that drone-captured footage can be analysed automatically using computer vision, reducing the need of human expose to dangerous environments. Early rust and cracks detection can prevent structural failures before they escalate.

Accessibility

RotorAI is deployed as a browser based web application requiring no installation or technical knowledge. This makes advanced defect detection accessible to broader range of users including contractors and non-technical inspectors who can upload images or use the live camera directly from their device.

Commercial Opportunity

RotorAI follows a B2B SaaS model where inspection companies can pay per analysis or a subscription model. The modular architecture also allows the system to be extended to support additional defect types or industries such as automotive, civil engineering and many more. Rather than replacing engineers entirely, it serves as a tool that reduces inspection time and cost while keeping human experts in the decision loop.

Risks

Despite its potential, several risks must be acknowledged. False negatives where real defects go undertected could have serious consequences if the system if fully relied upon without human intervention. The model's performance is also constrained by its training data which may not generalise well to different surface types. Any commercial deployment would need to address liability concerns and regulatory compliance being used in inspection workflows.

6.4 Future Work

- **Real-time 3D Gaussian Splatting reconstruction**
- **User authentication and history database**
- **Other defects types beyond rust and cracks**

7.0 Appendices

References

[1] RotorAI, "RotorAI Web Application," [Online]. Available: https://rotor-ai.vercel.app/. [Accessed: Apr. 2026].

[2] Nerfstudio, "Gaussian Splatting," Nerfstudio Documentation. [Online]. Available: https://docs.nerf.studio/nerfology/methods/splat.html. [Accessed: Apr. 2026].

[3] Nerfstudio, "Nerfstudio Documentation," [Online]. Available: https://docs.nerf.studio/. [Accessed: Apr. 2026].

[4] Polycam, "Gaussian Splatting Tool," [Online]. Available: https://poly.cam/tools/gaussian-splatting. [Accessed: Apr. 2026].

[5] Nerfstudio Project, "Nerfstudio GitHub Repository," [Online]. Available: https://github.com/nerfstudio-project/nerfstudio/. [Accessed: Apr. 2026].

[6] W. Kentaro, "gdown GitHub Repository," [Online]. Available: https://github.com/wkentaro/gdown. [Accessed: Apr. 2026].

[7] GraphDECO INRIA, "3D Gaussian Splatting GitHub Repository," [Online]. Available: https://github.com/graphdeco-inria/gaussian-splatting. [Accessed: Apr. 2026].

[8] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Transactions on Graphics*, 2023. [Online]. Available: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/3d_gaussian_splatting_high.pdf. [Accessed: Apr. 2026].

[9] Luma AI, "Luma Web Library," [Online]. Available: https://lumalabs.ai/luma-web-library. [Accessed: Apr. 2026].

[10] Nature, "Corrosion Detection Study," *npj Materials Degradation*, 2022. [Online]. Available: https://www.nature.com/articles/s41529-022-00232-6. [Accessed: Apr. 2026].

[11] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D Gaussian Splatting for Geometrically Accurate Radiance Fields," in *SIGGRAPH 2024 Conference Papers*, Association for Computing Machinery, 2024. [Online]. Available: https://surfsplatting.github.io/. [Accessed: Apr. 2026].

8.0 Appendices

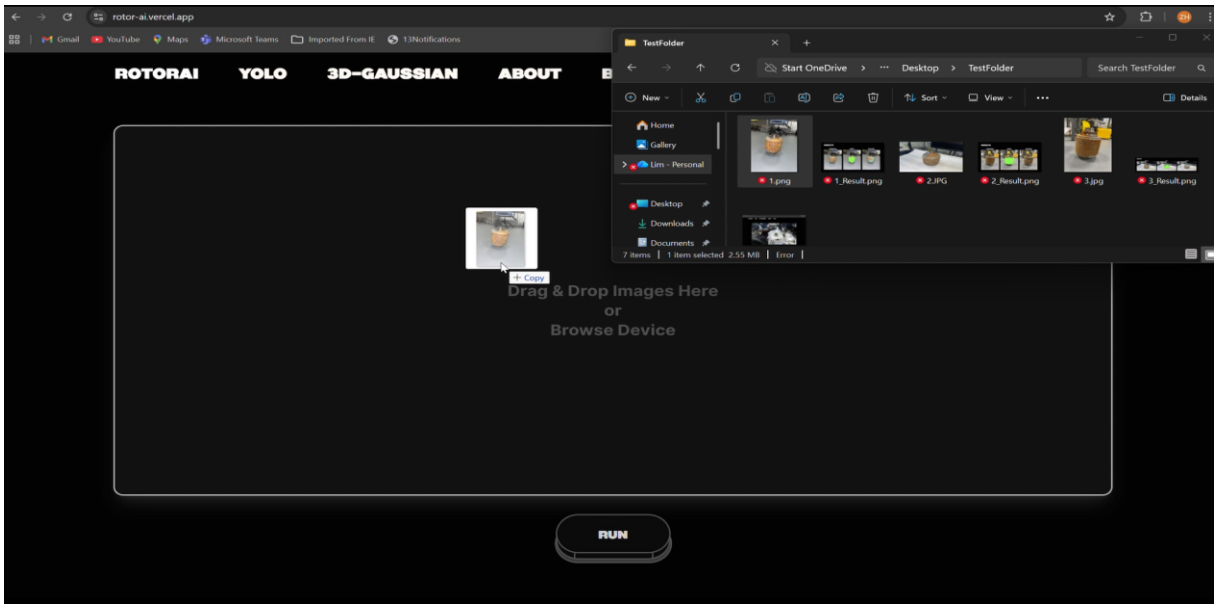
The following data can be downloaded from this github repository here:

https://github.com/ZEHUEI/RotorAI-zip

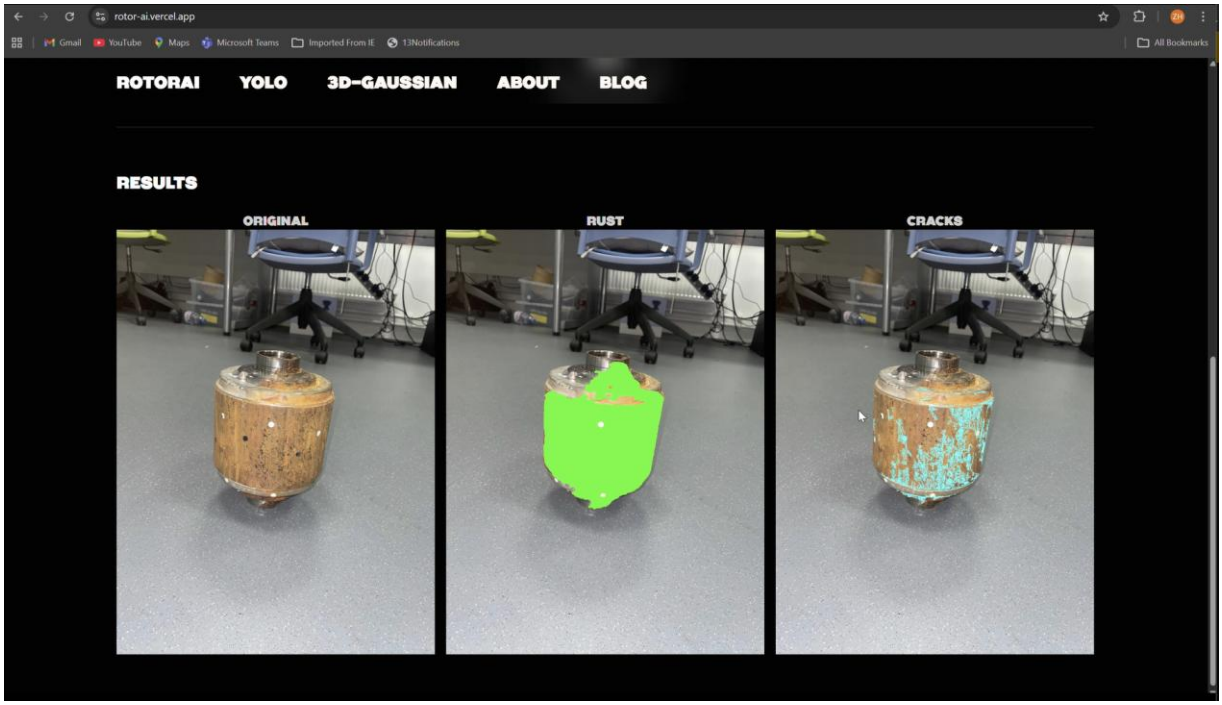
Appendix A – User Manual:

How to upload images and what formats are accepted? (RotorAI page)

Drag or browse from device

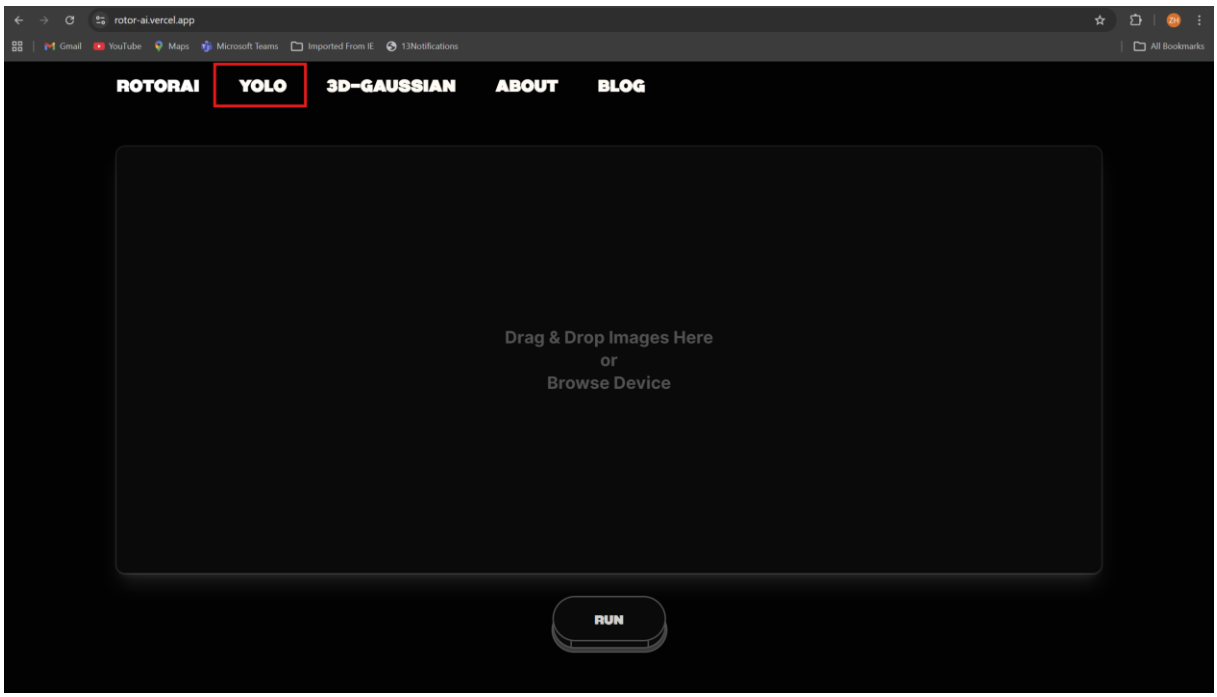


Click on the RUN button and wait for the Results will be below

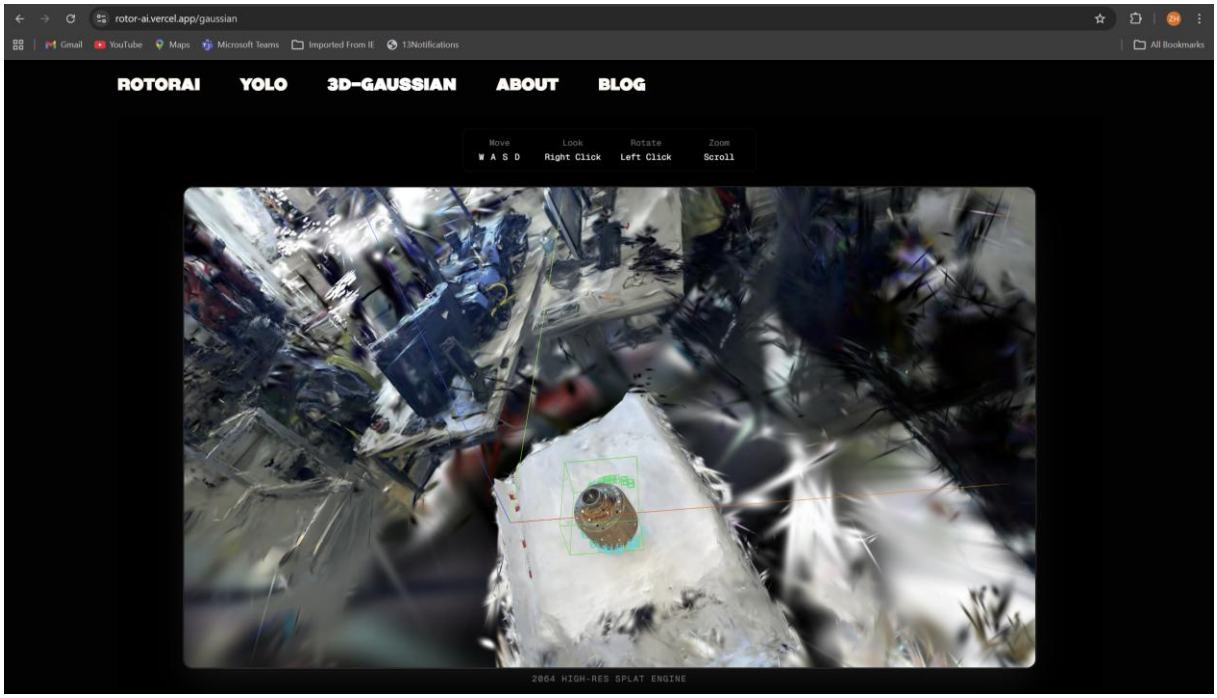


How to use live camera detection? (YOLO page)

Click on the YOLO and start the real-time video inspection



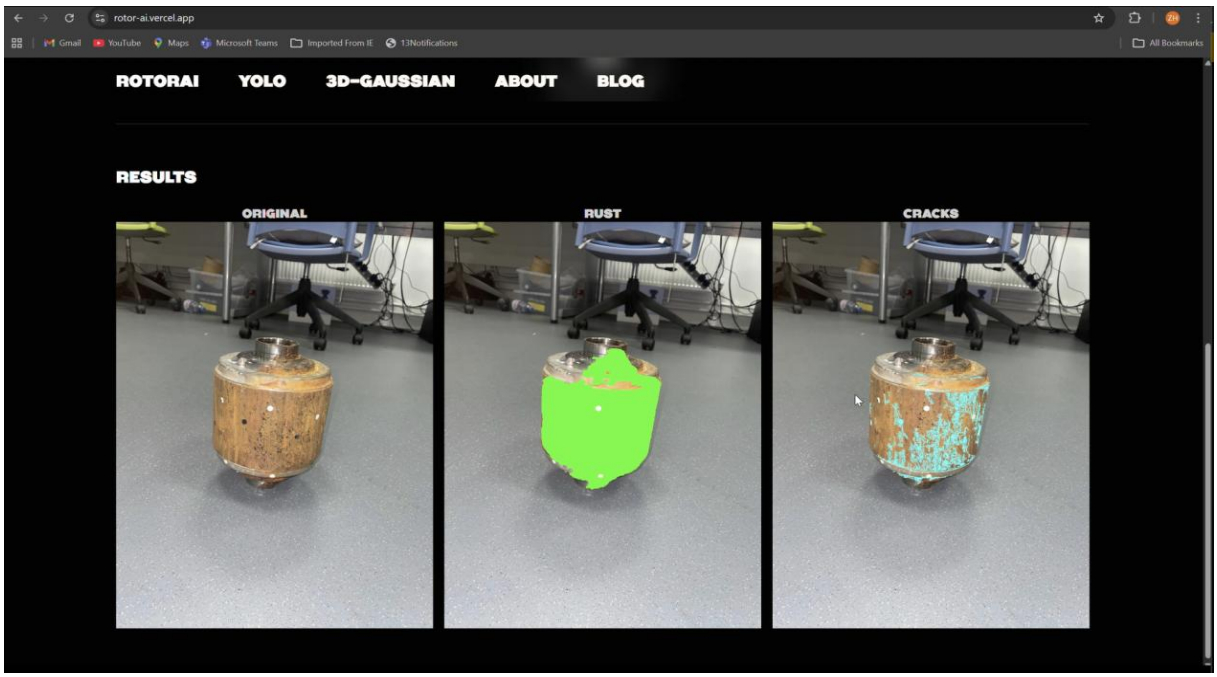
How to interact with 3DGS Viewer? (3DGS page)



WASD Keys to move around, Right Click to move camera, Left Click to rotate and Scroll wheel to zoom in /out.

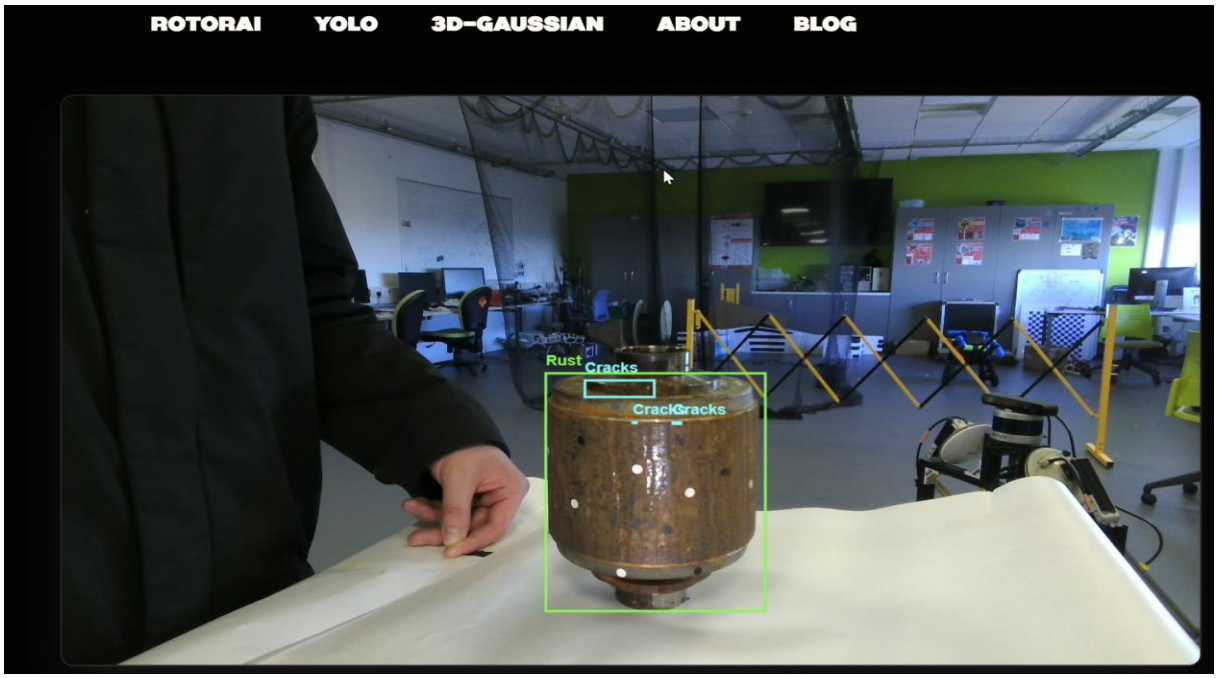
What output images means?

RotorAI page, Image processing:



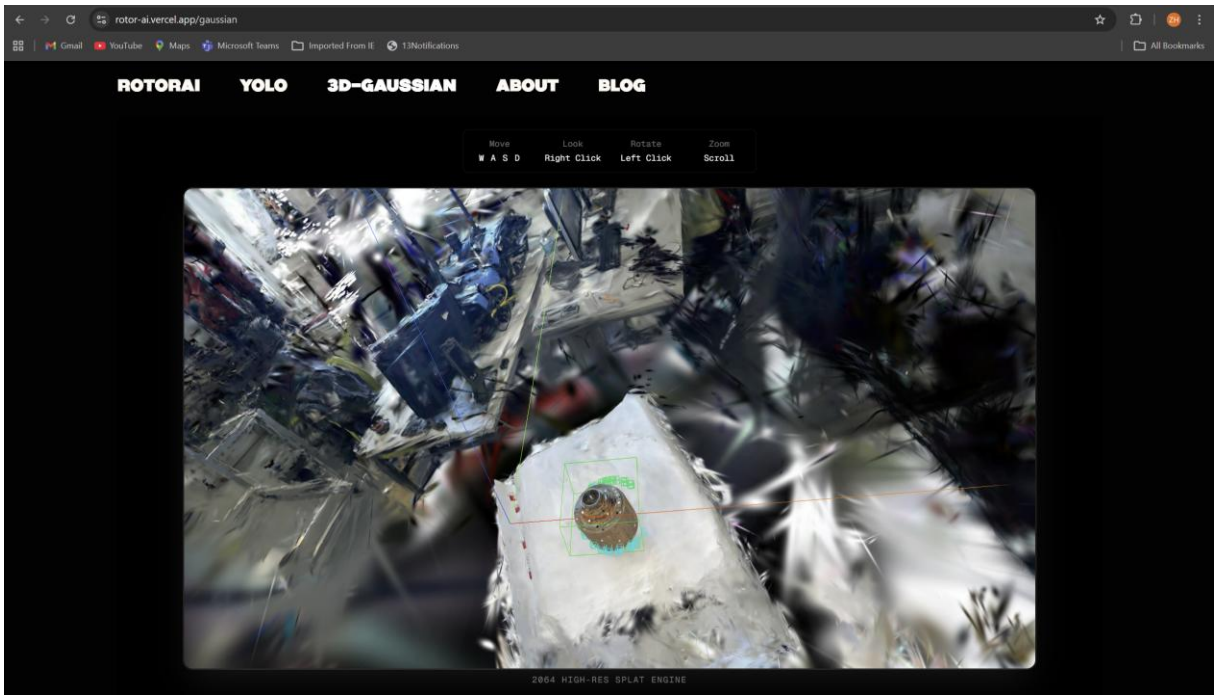
Shows original image to compare with rusts and cracks masks where rust and cracks are located**.**

YOLO page, Real-Time processing:



Shows bounding boxes of rusts and cracks, green boxes are rust and cyan boxes are cracks.

3DGS page, 3DGS viewer:

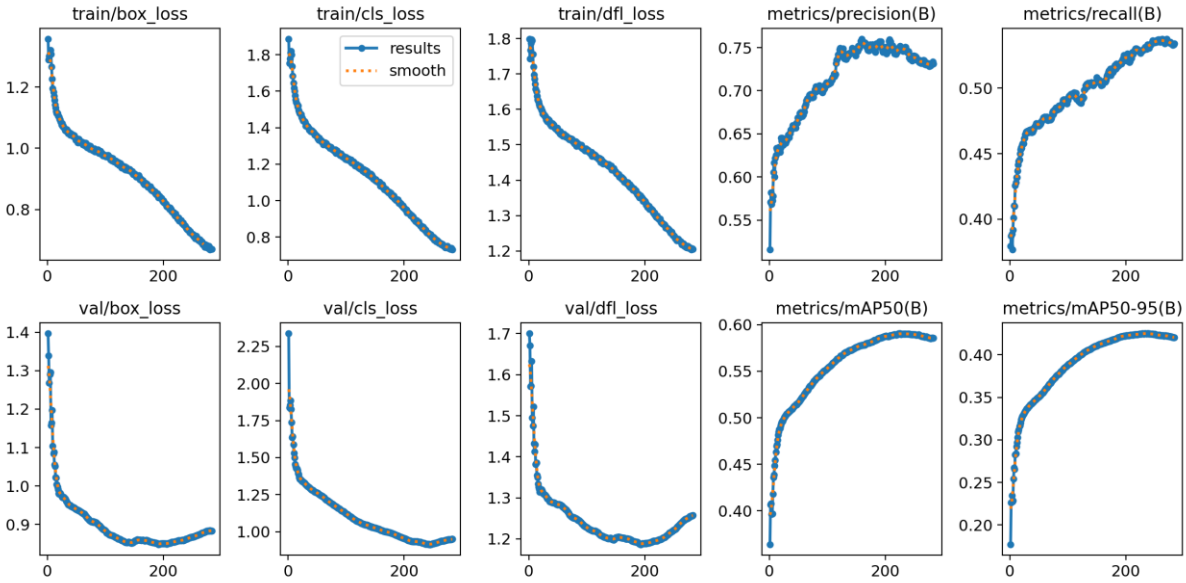


Green 3D boxes are where Rust are located and Cyan boxes are where cracks located.

Appendix B – Test Results:

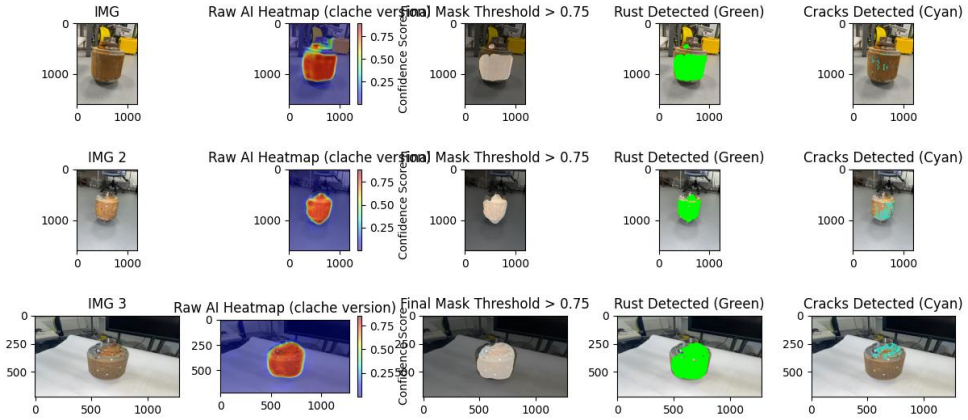
Model evaluation metrics:

YOLO Model:



TensorFlow Model:

Actual Testing Results:



Appendix C – Additional Resources:

Training configuration details:

TensorFlow:

- **U-Net with Segmentation_models**
- **Encoder: ResNet50**
- **Size: 786 x 768 x 3**
- **Adam Optimizer with Learning rate 5e-6**
- Custom fine tuned Loss Function
- **100 EPOCH**
    - **10x Dice Loss (Overlap quality)**
    - **5x Focal Loss (Class imbalance)**
    - **8x Binary Cross Entropy (Pixel learning)**

- Augmentations
    - Geometric
        - **Tranlsation ($\pm 10\%$)**
        - **Horizontal Flip**
        - **Zoom (-10%)**
        - **Rotation (3%)**
    - Photometric

        - **Contrast ($\pm 15\%$)**
        - **Brightness ($\pm 15\%$)**
        - **Hue Shift**
        - **Gamma variation (0.5 – 1.5)**
        - **Random Blur**

PyTorch (YOLOv8)

- **YOLOv8m model (Medium)**
- **300 EPOCH**
- **Image Size: 768**
- **Batch Size: 8**
- **Device: GPU (device = 0)**
- **Learning rate: 3e-4**
- **Scheduler: Cosine decay**
- **Warmup: 5 EPOCH**
- **Weight DecayL 5e-4**
- Augmentation
    - Spatial
        - **Mosaic = 1.0**
        - **MixUp = 0.15**
        - **Scale = 0.5**
        - **Rotation =** $\pm 15\ Deg$
        - **Shear =** $5\ Deg$
        - **Perspective = 0.0005**
    - Flips
        - **Horizontal = 0.5**
        - **Vertical = 0.1**
        - Colour (HSV)
        - **Hue = 0.015**
        - **Saturation = 0.7**
        - **Value = 0.4**